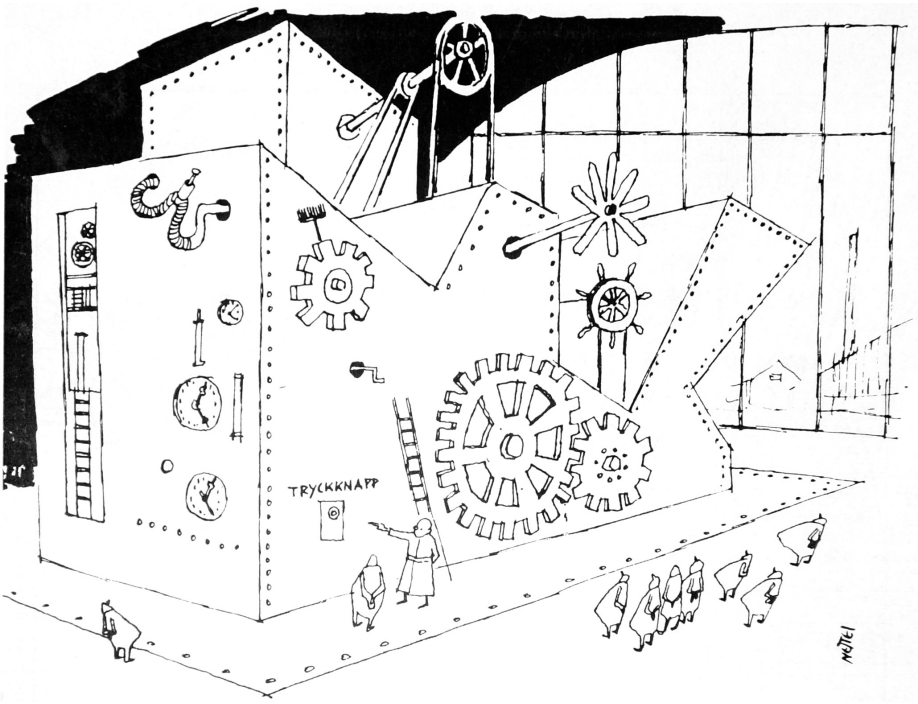
DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING,
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2021

A Modeling Language for Timed Automata

ERNST WIDERBERG



"If I push that button, this machine makes another machine like this machine."

KTH ROYAL INSTITUTE OF TECHNOLOGY
SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Cover illustration by Ejnar Nettelbladt

A Modeling Language for Timed Automata

Ett modelleringsspråk för tidsautomater

Author

Ernst Widerberg
ernstwi@kth.se

Supervisors

Jesus Mauricio Chimento
chimento@kth.se

Viktor Palmkvist
vipa@kth.se

Examiner

Cyrille Artho
artho@kth.se

Friday 26th February, 2021

Abstract

This work details the design and implementation of a modeling language for timed automata. The primary intended use of the language **TML** is as an interface to controller synthesis system **m2mc**, which is being developed in a current KTH/Chalmers research project.

TML is evaluated by a qualitative comparison with the modeling languages of two well-known model checking tools: Uppaal and Kronos. Two example systems (Fischer's mutual exclusion protocol and CSMA/CD) are implemented in all three languages, to discover the relative merits of each language.

Although not as feature rich as Uppaal, TML brings some new language features which are found to be potentially useful for modeling timed automata systems. These features are largely adopted from the general graph description language Dot, used by programs in the Graphviz software package.

As m2mc is still in early development and liable to change, an intermediate JSON representation for timed automata is defined. A compiler targeting this intermediate representation is implemented using **Miking**, a new compiler tool under development in a separate KTH project.

Further compilation from JSON to Uppaal is implemented as a proof of concept.

Sammanfattning

Detta arbete behandlar utformning och implementering av ett modelleringsspråk för tidsautomater. Språket **TML**:s huvudsakliga tänkta tillämpning är att fungera som ett användargränssnitt för kontrollsyntessystemet **m2mc**, vilket utvecklas i ett pågående forskningsprojekt på KTH och Chalmers.

TML utvärderas genom en kvalitativ jämförelse med modelleringsspråken för två välkända model checking-verktyg: Uppaal och Kronos. Två exempelsystem (Fischers protokoll för mutual exclusion och CSMA/CD) implementeras i vardera modelleringsspråk för att undersöka de olika språkens relativa fördelar och nackdelar.

Fastän TML inte är lika omfattande i funktionalitet som Uppaal så bidrar språket med en del nya funktioner, vilka baserat på utvärderingen anses kunna vara användbara för modellering av tidsautomatsystem. Dessa funktioner hämtas till stor del från språket Dot, vilket används i mjukvarupaketet Graphviz för att modellera generella grafer.

Eftersom m2mc är i tidig utveckling vore direkt integration med TML inte praktiskt användbart. Därför definieras istället ett mellanformat för tidsautomater i JSON. En kompilator för TML som producerar detta mellanformat implementeras med användning av **Miking**, ett nytt kompilatorverktyg under utveckling i ett separat KTH-projekt.

Som ett koncepttest implementeras vidare kompilering från JSON till Uppaal.

Contents

Chapter 1

Introduction

A timed automaton is a model of computation which is useful for analyzing the behaviour of real-time systems. There exists a number of model checkers which use timed automata, two of the most well-known being Kronos [17] and Uppaal [24]. These are tools which enable the verification of temporal logical properties on timed automata models. For instance, verification might ensure that some error state of the model is unreachable, or that a particular transition between two states occurs within a set time limit.

Model checkers are typically used to verify properties on model representations of existing systems. Naturally, the validity of safety guarantees acquired from model checking are dependent on the accurate correspondence between system and model. The translation from system to model is therefore a potential source of errors.

m2mc (Model to Machine Code) [9], a current research project at KTH and Chalmers, seeks to eliminate this source of errors by strengthening the connection between system and model via formally verified *controller synthesis*. Starting from a timed automata model with a set of verifiable safety properties, an executable program is synthesized, with safety properties proven on the model guaranteed to hold for the synthesized program.

The objective of this degree project is to develop a timed automata modeling language for use as an interface to m2mc.

A secondary objective concerns the chosen method of implementation. Writing a compiler from scratch can give great performance, but it is also generally a costly and quite involved process. A hand-written compiler can also be difficult to adapt and extend with new language constructs, as the implementation of any one construct is spread out across a set of compilation phases (typically at least parsing, semantic analysis, and code generation).

To address these issues various tools and methods for language implementation have been developed, such as parser generators, embedded languages, and metalanguages. A new effort in this latter category is being made in another current KTH research project: the **Miking** [8] metalanguage system. Miking presents a functional programming interface for compiler construction, with

extendability through the composition of smaller *language fragments* being a key feature.

In this degree project, the timed automata modeling language TML is designed as an interface to m2mc, and a compiler is implemented using Miking.

TML's design is based on a study of three related projects: Uppaal, Kronos, and Graphviz Dot. Uppaal and Kronos form the basis for evaluation of TML.

1.1 Research Questions

The main motivation for this work is to design a good interface to m2mc. A secondary motivation is to assess the usability of Miking as a tool for compiler construction.

TML is evaluated by a comparison with Uppaal and Kronos. We ask specifically whether the modeling languages used in Uppaal and Kronos respectively can be improved upon, and whether TML achieves any such improvement.

1.2 Scope

A complete interface for m2mc would need to support not only description of timed automata models, but also the expression of safety requirements for model checking. To avoid too broad a scope TML is focused on description, leaving model checking to be considered as an extension.

1.3 Societal Impact

As reliance on software grows in all parts of society, software correctness becomes ever more important. Serious adverse effects of malfunctioning software can range from economic loss to threats against health and safety.

Formal methods like model checking and controller synthesis are important tools for producing software with greater guarantees of correctness than what is possible using conventional testing. Their use, however, is often time-consuming and requiring of a great deal of expert knowledge. To increase the viability of formal methods across a wider range of uses, better tools are needed. Accordingly, this work is focused on improving the usability of a controller synthesis tool.

Although the concrete consequences of the software developed in this project are ultimately decided by its use, the general goal of supporting software correctness is considered an ethical good.

1.4 Outline

The remainder of the thesis is organized as follows. In Chapter 2, we give some background on timed automata and compiler construction. Chapter 3 presents three existing modeling languages for timed automata. Chapter 4 describes the TML language, and some aspects of its design and implementation process. In Chapter 5, TML is compared to Uppaal and Kronos by the modeling of two example systems. The results of this comparison are discussed in Chapter 6. Finally, in Chapter 7, we provide conclusions and present some potential future improvements of TML.

Chapter 2

Preliminaries

This chapter presents a background on timed automata, and their use in model checking and controller synthesis. Additionally, some standard concepts in compiler construction are presented.

2.1 Finite Automata

A finite automaton A is a model of computation consisting of *locations* connected by *edges*. There is one *initial location* and one or more *accepting locations*. A small example is given in Fig. 2.1.

Each edge is labeled with a *symbol* from some *alphabet* Σ. An automaton reads a sequence of symbols (i.e., a *word*) following at each point the edge corresponding to the symbol read. If automaton A after having read the entire word w ends up in an accepting location, we say that A *accepts* w. The set of all words accepted by A is called the *language* $L(A)$ of A.

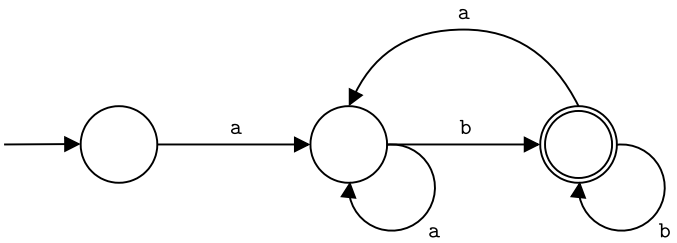


Figure 2.1: An automaton for the language `(a+b+)+`.

In a *non-deterministic* finite automaton, a location may have multiple outgoing edges with the same symbol. Such an automaton accepts a word w if there exists some run of the automaton on w such that it ends in an accepting state.

2.2 Timed Automata

Timed automata [2] are an extension of finite automata, with the concept of time represented as a finite set of real-valued clocks. Instead of recognizing a language of strings over some alphabet Σ, the language of a timed automaton is a set of *timed words*: pairs containing a symbol and a real-valued timestamp. Semantically, such a pair represents a symbol being read at a specific point in time.

Edges are extended with time constraints called *edge guards*, so that an edge may be used only during some real-time interval. Additionally, each edge has an associated set of clocks to be reset whenever the edge is traversed.

To this point we have described automata taking as input a predetermined sequence of symbols. In the model checking context it is more natural to consider an automaton as a dynamic system with many possible *behaviours*. In this view, the language of an automaton corresponds to all possible system behaviours. Instead of symbols it is common to say *actions* and instead of a symbol being read we say that an edge is *fired* and an action is *performed*.

2.2.1 Timed Safety Automata

In their original formulation [2], timed automata were derived from ω-automata, accepting or rejecting infinitely long timed words based on Büchi or Muller acceptance conditions. These acceptance conditions are both based on the concept of accepting states being visited infinitely often, which can be difficult to reason about. In particular, the implied conditions on surrounding non-accepting states are not always immediately obvious [5].

As an alternative formalism, *timed safety automata* (TSA) [15] adds *location invariants* to the timed automaton model. Location invariants, like edge guards, are simple conditions on clock values. For an automaton to enter or remain in a given location, the corresponding invariant must be satisfied. This added mechanism replaces Büchi or Muller acceptance conditions,[1] yielding a model of equal expressivity but which is simpler to work with.

We next give a precise definition of the TSA model, following the presentation in [5].

Syntax

Let C be an alphabet of real-valued clocks, and Σ be an alphabet of action symbols. As a convention we use $x, y, \ldots$ as names for clock variables, and $a, b, \ldots$ for actions.

A *clock constraint* is a conjunctive formula of the form $x * n$ or $x - y * n$, where $n \in \mathbb{N}$, $x, y \in C$, and $* \in \{\leq, <, =, >, \geq\}$. Let $B(C)$ denote the set of all clock constraints.

[1] Formally, all states are Büchi accepting [15].

A *timed automaton* A is a tuple $\langle N, l_0, E, I \rangle$ where

- N is a finite set of locations,
- $l_0 \in N$ is the initial location,
- $E \in N \times B(C) \times \Sigma \times 2^C \times N$ is the set of edges, and
- $I : N \to B(C)$ defines an assignment of location invariants.

Location invariants are restricted to a downwards closed form with one clock, i.e., $x \leq n$ or $x < n$.

As a shorthand for edges, $l \xrightarrow{g,a,r} l'$ is read as equivalent to $\langle l, g, a, r, l' \rangle \in E$.

Semantics

The semantics of timed safety automata are defined as a labeled transition system where states are pairs $\langle l, u \rangle$ of locations and clock assignments. In an *action transition*, an edge is fired and the automaton moves to a new location. This transition is atomic. There are also *delay transitions*, representing a moment during which the automaton stands still and allows time to pass.

In order to describe the transition system precisely some further definitions are required:

- A *clock assignment* u is a function $C \mapsto \mathbb{R}_+$ i.e., an assignment of a real value to each clock in C. Let $u \in g$ mean that with all clocks assigned according to u, the clock constraint g is satisfied.
- Addition $u + d$ of a clock assignment u with a non-negative real d is defined as the result of adding d to each clock in C, i.e., for each $x \in C, (u+d)(x) = u(x) + d$.
- $[r \mapsto 0]u$ is the clock assignment u with a set of clocks $r \subseteq C$ reset to zero, i.e., for all clocks $x \in C$,

$$([r \mapsto 0]u)(x) = \begin{cases} 0 \text{ if } x \in r, \\ x \text{ otherwise.} \end{cases}$$

Transitions are defined by the following rules:

- A *delay transition* $\langle l, u \rangle \xrightarrow{d} \langle l, u + d \rangle$ for some non-negative real d exists if $u \in I(l)$ and $(u + d) \in I(l)$.
- An *action transition* $\langle l, u \rangle \xrightarrow{a} \langle l', u' \rangle$ exists if $l \xrightarrow{g,a,r} l'$, $u \in g$, $u' = [r \mapsto 0]u$, and $u' \in I(l')$.

Note that since time is continuous, there are an infinite number of delay and action transitions.

2.2.2 Synchronizing Networks of Timed Automata

In modeling real-time systems it is often convenient to use several concurrently running automata. The TSA model does not define a way for automata to communicate, and this is thus left for individual software tools to design. Most use some type of special synchronization actions on edges to communicate between automata.

Note that adding communication does not increase the model's strict expressive power, as a network of communicating automata can be reduced to one automaton by a product construction.

2.3 Timed Automata Applications

This section gives an overview of how timed automata models are used in model checking and controller synthesis.

2.3.1 Model Checking

In considering the language of a timed automaton as a set of possible behaviours of some modeled real-time system, the purpose of model checking is generally to verify that some "good" behaviour is in this set, or that some "bad" behaviour is not.

Given a model of an elevator, for instance, you might want to verify that it can never move while the doors are open (bad behaviour) or that it can reach floor 19 within 30 seconds from any other floor (good behaviour).

A natural idea might be to express each such requirement r as a simple timed automaton V_r. The system model automaton S could then be verified by checking that $L(V_r) \subseteq L(S)$ for positive requirements, or that $L(V_r) \nsubseteq L(S)$ for negative requirements.

This is the language inclusion problem, which is decidable for standard finite automata but unfortunately not for timed automata [2]. For this reason model checking requirements are instead usually expressed as formulas in *timed computation tree logic* (TCTL), the checking of which is decidable for timed automata [1].

2.3.2 Controller Synthesis

Building on model checking, in controller synthesis we are not only interested in verifying safety properties on a model, but also in producing an executable *controller* to uphold safety properties on a system running in an adversarial environment. To grasp the details, we will need some concepts and terminology from the broader field of control theory.

Control Theory

Control theory concerns the study of systems which can be modeled in three discrete parts: *plant*, *controller*, and *environment* [7].

A plant is assumed to operate in an environment, from which it receives inputs which affect its state. The plant is modeled as a transition system (for our purposes, a timed automaton), with transitions defined for every possible input from the environment.

In addition to the environment, the plant can also receive inputs from a controller system. The controller's job is to monitor the state of the plant and steer it according to some *control law*.

Together, the plant and the controller form an autonomous system which reacts to environment changes.

As an example, we model an automatic climate control system as follows:

- *Plant* P: The state of a climate-controlled room, consisting of
  1. the temperature inside the room,
  2. a window which can either be open or closed, and
  3. a radiator which can either be running or not.
- *Environment* E: The world outside the room, which can affect the plant by changes in outside air temperature.
- *Specification* S: The temperature inside the room should always be between 20 and 25 degrees Celsius.
- *Control law* L (enacted by *Controller* C): When the inside temperature reaches above 24, open the window and turn off the radiator. When it falls below 21, close the window and turn on the radiator.
- *Actuator* A: An electrical component which can toggle the state of the window and radiator. Used by C to affect the state of the plant.

The temperature inside the room is affected by three factors:

F1 The temperature outside.

F2 The state of the window.

F3 The state of the radiator.

F1 is outside the control of C. Such a factor is called a *disturbance*. F2 and F3 are controlled by C (via the actuator A).

To enable modeling the plant, environment, and controller as three separate entities, the edges of the plant are marked as either *uncontrollable* or *controllable*. This defines which state changes can be induced by the environment vs. the controller, i.e., which inputs are disturbances and which are actuator signals [16].

The Controller Synthesis Problem

In control theory terms, assume that we can divide a real-time system model into environment E, plant P, and controller C. In order to model check this system, we need a complete understanding of the behaviour of all of its parts. This is called a *closed system* [16].

With $C(E, P)$ denoting a system consisting of plant P running in environment E under the influence of controller C, the **model checking problem** for a closed system can be phrased as:

> Given an environment E, a plant P, a controller C, and some requirement ϕ, does $C(E, P) \vDash \phi$ hold?

In an *open system* we have an environment and a plant, but the controller is undefined. In this setting, the **control problem** is

> Given E, P, and ϕ, is there a C such that $C(S) \vDash \phi$ holds?

Finally, the **controller synthesis problem** is to produce a witness for a positive answer to the control problem, that is to produce the controller C such that $C(E, P) \vDash \phi$ holds.

All the components in a controller synthesis system and their internal dependencies are illustrated in Fig. 2.2 below.

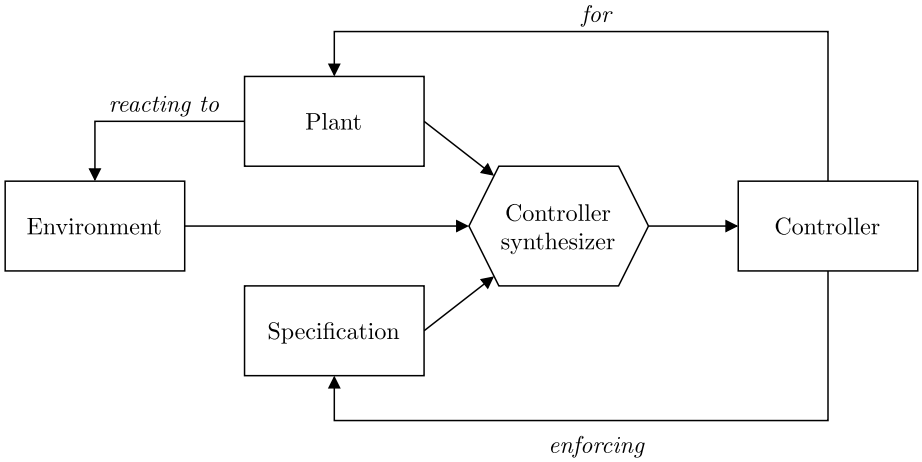


Figure 2.2: Overview of the controller synthesis problem.

An alternative way of looking at the controller synthesis problem is in terms of *timed games* [16]. In this framework, we view the system as a game between the controller and an adversarial environment. The controller synthesis problem then becomes the problem of finding a winning strategy for the controller to achieve some objective, given any play by the environment.

2.4 Compiler Construction

This section presents briefly some standard concepts used in compiler construction. A longer discussion is given to the topic of syntactic sugar, which is important for the design and implementation of TML.

2.4.1 Compiler Phases

A compiler is a program which translates code in a *source language* to some other *target language*. Usually the source language adds a level of abstraction over the target language, with the intent of making it easier to write.

Due to their long-standing ubiquity in computing, the implementation of a typical compiler (including the TML compiler) can usually be described in terms of some well-established phases.

Starting from source code viewed as a sequence of characters, *lexing* is the process of dividing this sequence into discrete *tokens*. The resulting token sequence is via *parsing* translated to an *abstract syntax tree* (AST), the branches of which represents program statements and expressions.

Various semantic analysis and transformation operations may be performed on the AST. Commonly occurring are *type checking*, *name analysis* (in which e.g., uses of variables are connected with their declarations), and optimization. Finally, the *code generation* phase uses the processed AST to produce output in the given target language.

2.4.2 Syntactic Sugar

In defining the concrete syntax of a programming language, it is common to make a distinction between core language features and syntactic sugar [23]. The largest part of the compiler – semantic analysis, AST transformations, and code generation – operate on an abstract syntax tree consisting of constructs from the smaller *core language*. On top of the core language, syntactic sugar is added to support new convenient syntax constructions which correspond more or less directly to existing elements of the core language. In an early compiler phase (just after parsing) known as "desugaring," statements in the full surface language are translated to the core language, i.e., the syntactic sugar is removed. This allows the addition of convenience features while leaving the majority of the compile chain unmodified, freeing the compiler from added complexity.

For a concrete example, we consider Miking's implementation of tuples as syntactic sugar for records [20].

Records should be familiar to the reader (perhaps as "structs") as a collection of named fields:

```
{ age = 28, name = "Ernst" }
```

A tuple is a fixed-size ordered collection of (unnamed) values:

```
(28, "Ernst")
```

Tuples are implemented as syntactic sugar for records via a simple translation, in which element n of the tuple becomes a field with the name n. After desugaring, the above tuple `(28, "Ernst")` becomes record `{ 0 = 28, 1 = "Ernst" }`.[2] Semantically, these two snippets of Miking code are completely equivalent.

Inherent to the concept of syntactic sugar is the introduction of slightly differing representations of a program. After desugaring, the compiler's view of a program is not the same as that of the user who wrote it. If not carefully considered, this situation can lead to some trouble.

One complication which arises is in error messages and debugging. Without special consideration, errors detected during a later compiler phase such as semantic analysis (performed on a desugared abstract syntax tree) results in error messages referencing the desugared program instead of that which the user actually typed. For example, if the first element of `(28, "Ernst")` is used in an incorrect type context, an error message might tell the user that field 0 has the wrong type.

The simplest solution to this issue is to make the correspondence between tuples and records part of the language definition, and expect the user to be aware of it. In this case no special action needs to be taken, but the cognitive burden on the user is increased. The tuple feature becomes what is known as a leaky abstraction.

A better solution, and indeed one which is normally used, is to reconstruct the original program from a desugared version, enabling the compiler to give error messages referencing the program as written. This process is called *resugaring*.

To achieve this, the compiler must somehow remember the original program form – the information contained in the difference between `(28, "Ernst")` and `{ 0 = 28, 1 = "Ernst" }` cannot be fully discarded.

A quick solution in this particular case could be to disallow record field names starting with digits in the surface language. We would then know that any record with field names beginning with $0 - 9$ was transformed from a tuple. In more complicated cases it can be necessary to store information for desugaring explicitly.

[2] In Miking, record field names can consist of arbitrary strings. However, field names such as "0" which do not parse as identifiers must be escaped using the syntax `#label"0"`. Accordingly, the actual syntax for this particular record is `{ #label"0" = 28, #label"1" = "Ernst" }`. This detail is ignored for clarity in the main text.

2.5 Summary

A timed automaton is a model of computation useful for representing real-time systems. Timed automata can be used for model checking and controller synthesis. The aim of model checking is to prove safety properties on a model. In controller synthesis, an executable program is produced to uphold safety requirements.

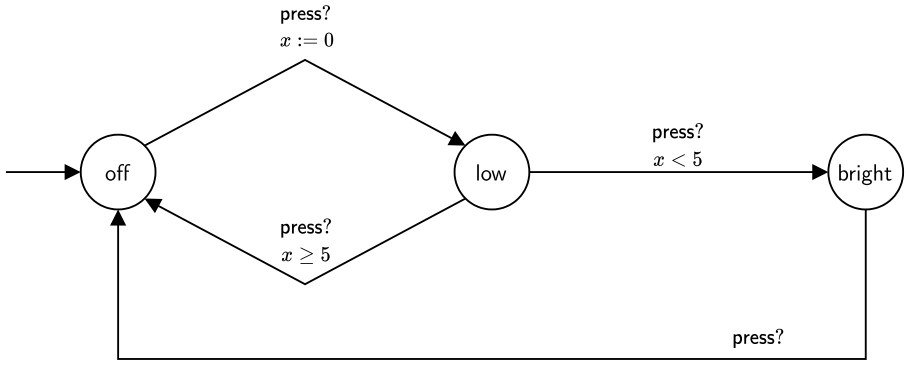
The mechanics of a compiler can generally be described as a sequence of phases. From a sequence of characters in some source language, and abstract syntax tree (AST) is produced. The AST is used to produce code in a target language. A commonly used strategy for simplifying the implementation of compilers is to treat some language features as syntactic sugar. This strategy may be employed wherever there exists a close conceptual resemblance between two different syntax constructions.

Chapter 3

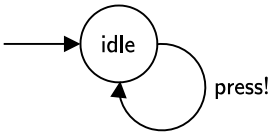
Related Work

This chapter presents a set of existing languages for constructing timed automata models. A study of these languages forms the context in which TML is created, influencing its design and feature set. Additionally, Uppaal and Kronos serve as the basis for TML's evaluation in Chapter 5.

The selection of Uppaal and Kronos for inclusion in this study is based on their relevance in the field, as indicated by [27], [6], and [19]. The third language, Graphviz Dot, is included even though it is not specifically related to timed automata, as some of its features are used more or less directly in TML's design.



(a) Automaton L representing a lamp with two brightness settings.



(b) Automaton U representing a user of L.

Figure 3.1: A simple timed automata network N.

For illustration purposes an example system N from [4], illustrated in Fig. 3.1, is used throughout this section. In this figure, inter-automata communication is pictured using Uppaal's version of synchronization actions (further explained in Sec. 3.2.1), where input action press? synchronizes with output action press!.

Automaton L models a lamp with two brightness settings controlled using a single button. When pressed twice in rapid succession, the highest brightness mode is activated. U models a user who may press the button at any time.

3.1 Graphviz Dot

Graphviz [13] is a collection of programs which operate on general directed or undirected graphs. Most Graphviz programs produce some kind of visualization, but there are also tools in the collection which perform simple graph manipulations like extracting the connected components of a graph. A common modeling language, Dot [14], is used across all Graphviz programs.

The lamp automaton L is modeled in Lsts. 3.1, 3.2 below. Note that the location and edge properties used here are not chosen to be understood by Graphviz tools. The Dot language allows for arbitrarily named properties, leaving it to individual software tools to assign semantic meaning to specific property names. The graph visualization program `dot`, for instance, (part of the Graphviz program suite, not to be confused with the Dot language) looks for `shape`, `label`, and `color` on location statements.

Listing 3.1: Dot implementation of automaton L from Fig. 3.1a.

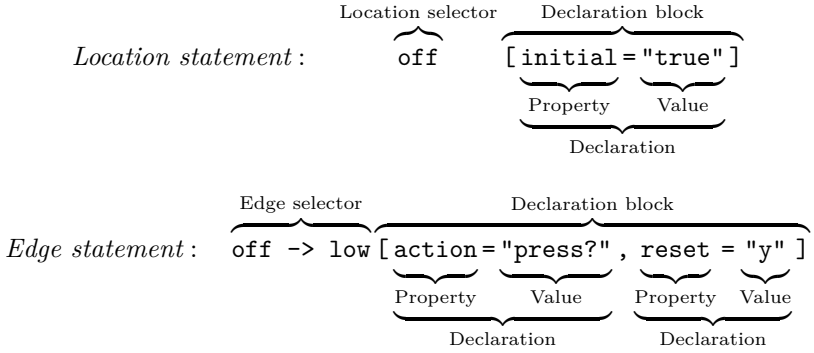
```
1  digraph lamp {
2      off [initial = "true"];
3      low;
4      bright;
5
6      off -> low    [sync = "press?", reset="x"];
7      low -> off    [sync = "press?", guard="x >= 5"];
8      low -> bright [sync = "press?", guard="x < 5"];
9      bright -> off [sync = "press?"];
10 }
```

Listing 3.2: Dot implementation of automaton U from Fig. 3.1b.

```
digraph user {
    idle         [initial = "true"];
    idle -> idle [sync = "press!"];
}
```

A Dot program represents a directed or undirected graph, as determined by the `digraph` (vs. `graph`) keyword on line 1 of Lsts. 3.1, 3.2. The program consists of a sequence of declarations of locations and edges, each with an optional set of associated properties. This type of statement is recurrent in graph modeling

languages (including TML), so we introduce some precise terminology adapted from the CSS language [10].



Semantically, a location statement in Dot will either create a new location (if none exists using the given identifier), or it will modify an existing location's properties. An edge statement always creates a new edge, as multiple edges can exist between the same two locations.

We next give brief descriptions of two useful Dot language features, variants of which are implemented in TML.

3.1.1 Default Properties and Subgraphs

When modeling many locations or edges which share the same properties, default properties can be used either globally or locally within a delimited code block. Such a block is called a *subgraph* in Dot, though it is not strictly a subgraph – it may for instance be composed exclusively of edge statements. The semantics of default properties are exemplified in Lst. 3.3 below. The corresponding graph is illustrated in Fig. 3.2. Note that in Dot terminology locations are called "nodes," which is reflected in the code.

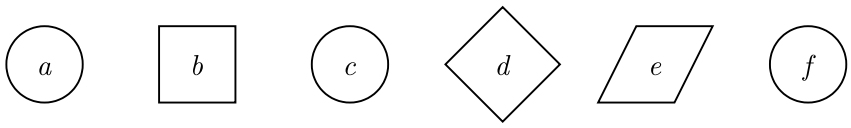


Figure 3.2: Graph produced by the program in Lst. 3.3.

Listing 3.3: Dot program illustrating the use of default properties.

```
digraph example {
    a;
    // Default applying to all subsequent nodes
    node [shape = square];
    b;
    // Resetting the default shape to circle
    node [shape = ""];
    subgraph {
        c;
        // Applies to all subsequent nodes in the subgraph
        node [shape = diamond];
        d;
        // Overriding a default with a local property
        e [shape = parallelogram];
    }
    f;
}
```

3.1.2 Edge Selector Shorthands

Subgraphs can be used in edge selectors to conveniently create multiple edges at once: `{ a, b } -> { c, d }` produces the graph below.

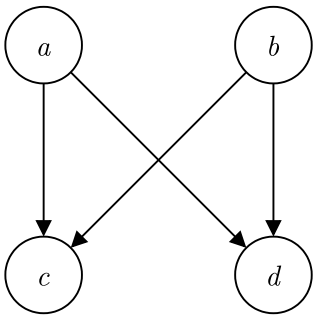


Figure 3.3: A graph defined by use of subgraphs in edge selectors.

An edge selector may also contain more than two endpoints, taking the form of a longer chain. Each point in the chain may consist of either a single location or a subgraph: `{ a, b } -> c -> { d, e }` produces the graph in Fig. 3.4. Properties defined on such a chain selector applies to each generated edge.

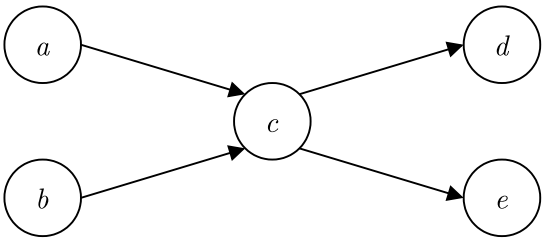


Figure 3.4: Edges defined using a chain selector with subgraphs.

3.2 Uppaal

Uppaal [24] is a timed automata model checking tool developed by researchers at Uppsala University and Aalborg University in Denmark. It is prominent in the field [6, 19, 27] and has been developed continuously since 1999 [24].

The core functionality of Uppaal is model checking, but there exists many spin-off projects developed by the same team, including Uppaal Tiga [26] for controller synthesis (of the timed games variant).

Real-time systems in Uppaal are modeled as networks of timed automata based on an extension of the timed safety automata (TSA) formalism. Model checking requirements are expressed in a subset of TCTL.

Our main interest is in the model construction functionality, which is quite advanced and can be used via three different interfaces. Uppaal's primary interface is a GUI, which includes support for both modeling and verification. Timed automata are constructed by placing locations on a canvas and connecting them with edges using a mouse.

Before the implementation of the GUI, modeling was entirely text-based using a custom modeling language XTA. XTA is still supported and can be used interchangeably with the GUI. Some elements of the modeling interface remains as XTA text input fields in the GUI, such as the definition of edge guards and synchronization actions.

A third interface to Uppaal is a Java API. This interface is perhaps the most powerful, but it is also more complicated – at least to users without prior experience with Java.

In the following we focus on XTA, since it is most similar to TML. Uppaal's three interfaces are equivalent in that they all use the same underlying modeling system – there is feature parity between the GUI, XTA, and the Java API.

3.2.1 Modeling with Uppaal XTA

This section presents an overview of Uppaal's timed automata modeling semantics and the XTA syntax. The semantics are based on TSA, so e.g., location invariants and edge guards should be assumed without explicit comment. The presentation is restricted to a subset of modeling features, with focus on what is relevant for the remainder of the thesis.

Listing 3.4: Uppaal XTA implementation of automata network N from Fig. 3.1.

```
1  chan press;
2
3  process Lamp() {
4      clock x;
5      state off, low, bright;
6      init off;
7
8      trans
9          off -> low {
10             sync press?;
11             assign x = 0;
12         },
13         low -> off {
14             guard x >= 5;
15             sync press?;
16         },
17         low -> bright {
18             guard x < 5;
19             sync press?;
20         },
21         bright -> off {
22             sync press?;
23         };
24 }
25
26 process User() {
27     state idle;
28     init idle;
29     trans
30         idle -> idle {
31             sync press!;
32         };
33 }
34
35 system Lamp, User;
```

Templates

On a high level, automata in Uppaal are defined by *templates*, which are used to create concrete automata processes. Lst. 3.4 contains definitions of two templates Lamp and User. The *system definition* on line 35 instantiates these templates into processes.

Templates may have parameters, which are used to create multiple processes from a single template. If a template with free parameters is used in a system definition, Uppaal automatically creates one process per combination of possible parameter values. In the example below, a is a bounded integer with the possible values $\{1, 2\}$, and b is a Boolean. The instantiation of template P on line 4 creates four processes: $P(0, \mathsf{true})$, $P(0, \mathsf{false})$, $P(1, \mathsf{true})$, and $P(1, \mathsf{false})$.

```
1 process P(int[0,1] a, bool b) {
2     ...
3 }
4 system P;
```

Templates may also be partially or fully instantiated to create other templates, with some or all parameters bound:

```
process P(int[0,1] a, bool b) {
    ...
}
Q(const int[0,1] a) = P(a, true);
R = P(0, false);
system Q, R;
```

The system above consists of $Q(0)$, $Q(1)$, and R, which is equivalent to $P(0, \mathsf{true})$, $P(1, \mathsf{true})$, and $P(0, \mathsf{false})$.

Synchronization

Automata processes synchronize by *input* and *output actions* over *channels*. For a channel a, an edge labeled with input action $a?$ may be fired only in synchronization with another process firing an edge with the corresponding output action $a!$. In Lst. 3.4 one channel press is used to model the sending and receiving of a user's button press between two processes. The channel is declared globally, as is the case in all practical uses of channels.

Note that each edge in a synchronized transition must be *enabled* at the time it is fired, meaning that its guard must be satisfied and the automaton must be in the appropriate location.

Processes can also synchronize over one-to-many *broadcast channels*. An edge with output action $b!$ over a broadcast channel b may fire at any time. Any other processes with enabled $b?$-labeled edges will fire simultaneously, though unlike for binary synchronization actions there is not any requirement that such an edge is enabled or even exists for $b!$ to be performed. This feature is useful for modeling events that are one-way dependent. As an example, say a train

is departing at a set time by firing an edge labeled $b!$. Any passenger which happens to be on board at that time can synchronize with $b?$, though the train will depart regardless.

Although in Uppaal input and output actions are the only type of actions which are visible, formally any edge without a synchronization action label is implicitly assigned the internal action τ. The total set of Uppaal actions is defined by $\Sigma_i \cup \Sigma_o \cup \{\tau\}$, where Σ_i is the set of input actions and Σ_o output actions.

Urgent and Committed

A channel may be declared *urgent*, meaning that a possible action over it cannot be delayed. There are also urgent locations, for which delay transitions are not permitted, i.e., no time can pass while a process is in an urgent location. This is a stricter requirement than that of an urgent channel – if there is no enabled outgoing edge from an urgent location l at the time l is entered then the process will deadlock. Formally, a timed word which would induce such behaviour is not in the automaton's language.

Even more restrictive are *committed* locations. Like with urgent locations, no time may pass in a committed location. Additionally, while any process in the system is in a committed location, only transitions from committed locations are allowed globally.

Expressions

A powerful feature of Uppaal is that all properties on locations and edges – location invariants, edge guards, synchronization actions, update statements, etc. – are defined using arbitrarily complex expressions in a C-like syntax.

Expressions may include references to process-local or global variables and functions, which can be defined over most Uppaal types (e.g., clocks, channels, and integer variables). A type checker enforces some rules to make this work; for example an expression e used in output action label $e!$ must evaluate to a channel and must be side-effect free.

This feature enables parts of a system which are not subject to model checking to be expressed using standard programming constructs instead of automata. A typical use would be the implementation of supporting data structures such as FIFO queues or linked lists.

3.3 Kronos

Another well-known model checker for timed automata is Kronos [28], developed at French research institute Verimag in the late 1990s [6]. The most recent release is from 2002 [17].

Kronos uses an automata model which is largely equivalent to TSA, except that an edge may carry more than one action. Semantically, an edge firing triggers all its associated actions simultaneously.

Synchronization actions are used to model communication between automata in a network. Unlike in Uppaal, there is no distinction between input and output actions. An automaton which uses a synchronization action a is said to *synchronize on* a. An edge labeled with a may fire only if every other automaton in the network which synchronizes on a simultaneously fires an a-labeled edge. Synchronization actions in Kronos can thus be used for either one-to-one or many-to-many communication. The difference with Uppaal is meaningful when modeling simultaneous synchronization between three or more automata.

We note that this is the only means of communication between automata; i.e., there are no shared clocks as exists in Uppaal. There are also no variables or functions, shared or otherwise.

Besides synchronization actions, internal actions are supported as well. They serve no mechanical purpose but can be used as a kind of symbolic annotation.

Listing 3.5: Kronos implementation of automaton L from Fig. 3.1a.

```
#locs 3
#trans 4
#clocks x
#sync press

loc: 0
prop: init off
invar: TRUE
trans:
TRUE => press; x := 0; goto 1

loc: 1
prop: low
invar: TRUE
trans:
x >= 5 => press; ; goto 0
x < 5 => press ; ; goto 2

loc: 2
prop: bright
invar: TRUE
trans:
TRUE => press; ; goto 0
```

Listing 3.6: Kronos implementation of automaton U from Fig. 3.1b

```
#locs 1
#trans 1
#clocks
#sync press

loc: 0
prop: init idle
invar: TRUE
trans:
TRUE => press; ; goto 0
```

An example of Kronos's modeling language is given in Lsts. 3.5, 3.6 above. Each automaton in a network is defined using one text file following a rather strict and straightforward syntax. The file starts with four lines describing some general properties of the automaton, namely its number of locations and edges, as well as a listing of all clocks and synchronization actions used. This is followed by one code block for each location l, containing in order:

1. `loc`: A location number, which is used as l's identifier in edge definitions (see `trans` below).
2. `prop`: A set of identifiers representing Boolean propositions (used in model checking) which are set to true when the automaton is in l. The proposition `init` carries special meaning, indicating l as the initial location.
3. `invar`: A location invariant formula following the TSA definition of clock constraints, i.e., without the restriction to a downwards closed form using one clock. As a formula following `invar` is syntactically required, the value `TRUE` is used to symbolize the absence of a location invariant.
4. `trans`: A listing of outgoing edges from l, on the format `<g> => <id> ... <id> ; <r>, ... , <r> ; goto n`, where
   - `<g>` is a guard,
   - `<id> ... <id>` is a space-delimited set of actions,
   - `<r>, ..., <r>` is a comma-delimited list of clock assignments, where the right side is either zero or another clock identifier[1], and
   - `n` is the target location number.

Model checking requirements, not pictured in Lsts. 3.5, 3.6, are expressed using TCTL formulas.

3.4 Summary

This chapter presented three software tools related to timed automata: Graphviz Dot, Uppaal, and Kronos. Dot is a general graph modeling language. Uppaal and Kronos are model checking tools for timed automata, with their own respective modeling languages. Both support modeling networks of timed automata which communicate using synchronization actions, though they differ in semantic detail.

[1] Compare with TSA, where an edge can only reset clocks to zero. In Uppaal, any integer expression is allowed on the right-hand side of a clock assignment.

Chapter 4

The TML Language

This chapter concerns the design and implementation of TML. In Sec. 4.1, some initial challenges and resulting design choices are presented. The core TML language and some optional extension features are described in Sec. 4.2. The full syntax is given in Sec. 4.3. Sec. 4.4 gives some technical details on the implemented compiler.

4.1 Design Challenges

With m2mc still in early development, developing a modeling language to fit its needs comes with some challenges related to underspecification, both in design and in technical details.

C1 Like Uppaal and Kronos, m2mc is being developed with the TSA model of timed automata as a formal foundation. Beyond this, there is not yet a complete definition of its intended feature set. Some mechanism for communication, for example, is likely be considered at some point, though this addition is not certain nor is it determined how such a feature would work semantically.

C2 The technical framework around m2mc is also liable to change, i.e., the programming language used in its implementation. Current work is being made in the Coq proof assistant, but this will likely change later on in development. Taking this into consideration, TML must be possible to integrate in as wide as possible a selection of potential technical frameworks.

Intermediate Representation

Ideally, this degree project would include integration of TML into the larger m2mc system, by compilation of TML source code all the way to a timed automata representation internal to m2mc. Unfortunately this is not practical given the early development stage of m2mc (challenge C2).

To counter this problem, an intermediate representation (IR) of timed automata is defined and used as a target format for the TML compiler developed in this project. The IR is encoded in the widely supported JSON format, to support easy integration with m2mc regardless of future technical choices.

For demonstration purposes, translation of the JSON IR to Uppaal XTA is implemented in a JavaScript program completely separate from the TML compiler (see further Sec. 4.4.3).

Fig. 4.1 shows the components of the resulting system, with two different potential back-ends. Squares represent data, and circles represent software. Software which is implemented in this project is circled in dotted red. For the entirety of this thesis, "the TML compiler" refers to program (a) in this figure.

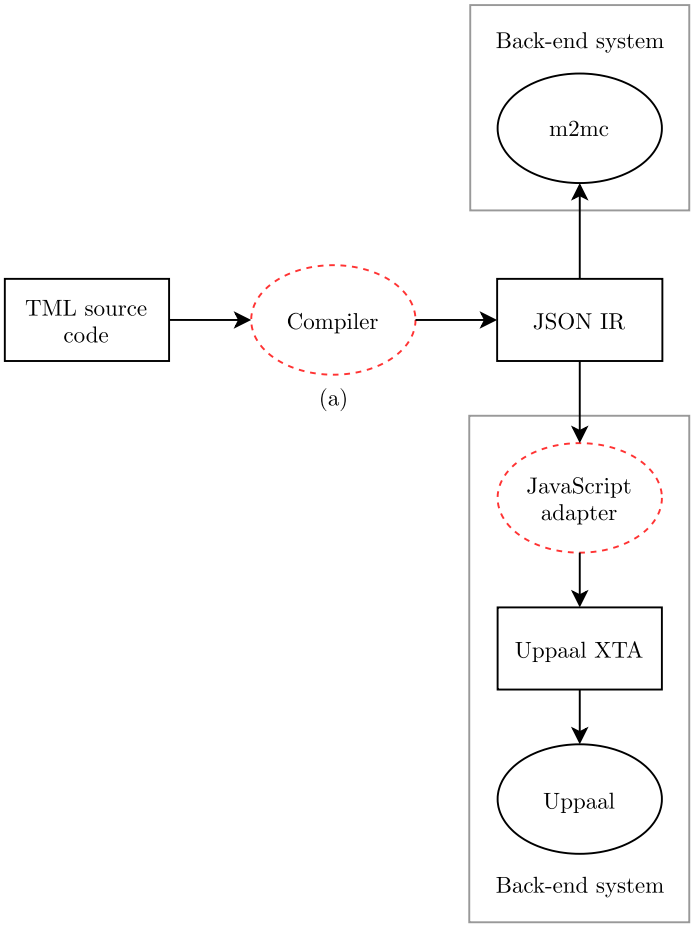


Figure 4.1: Overview of TML integration with m2mc and Uppaal.

Shallow vs. Deep Features

To address challenge C1, we define a distinction between *shallow* and *deep* language features.

Shallow features are considered fully compatible with the basic TSA model of timed automata. A shallow feature is either directly a part of the TSA model, or it can be implemented as light syntactic sugar on top of it.

Deep features are dynamic in nature, and cannot cleanly be statically resolved to fit TSA. They thus require extension of the JSON IR and explicit support from any back-end system using it. A deep feature we have seen in Sec. 3 which is present in most timed automata tools is support for networks of automata via inter-automaton communication, using e.g., synchronization actions or shared integer variables.

To make the distinction more precise, we consider as "deep" any syntactic sugar feature which creates a need for extensive resugaring in the back-end system. For example, inter-automata communication could be implemented as syntactic sugar on top of TSA using a product construction, but the distance between source model and desugared TSA model is determined too large to make this a practical choice.

In fact, resugaring is not considered at all in this project, so all shallow features must work well without any resugaring. This imposes the requirement that semantic errors in syntactic sugar constructs must be caught by the TML compiler, and not carried through to a back-end system.

Since m2mc has not committed to supporting any feature outside the bounds of TSA, a decision is made to focus on shallow features. The deep features which are added are done so on a hypothesis of what might be useful to m2mc in the future. Deep features are implemented with clear separation from the core language, in a way which makes them easy to enable or disable.

4.2 Design

This section presents an overview of the TML language. First, the general structure of a TML program is explained, after which follows a detailed listing of shallow and deep features. The exact syntax of TML presented in Sec. 4.3.

As a starting example, L^T (Fig. 4.2) is implemented in Lst. 4.1 below. L^T is the lamp automaton L from Sec. 3 modeled without synchronization.

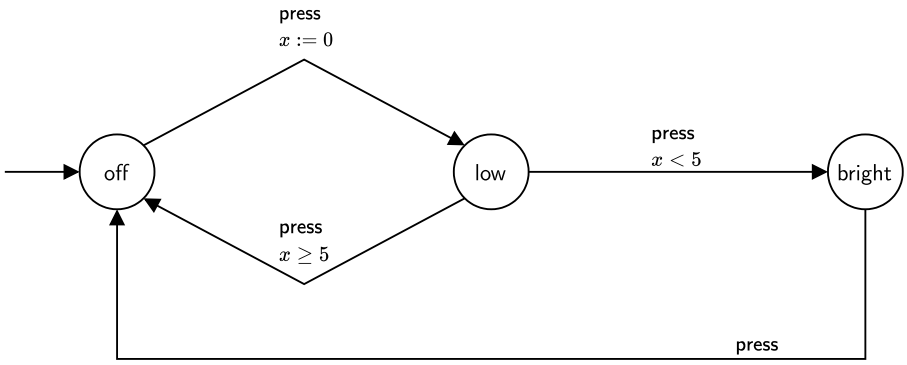


Figure 4.2: Automaton L^T representing a lamp with two brightness settings.

Listing 4.1: TML implementation of automaton L^T from Fig. 4.2.

```
init off
low
bright

off -> low
    action { press }
    reset { x }

low -> off
    guard { x >= 5 }
    action { press }

low -> bright
    guard { x < 5 }
    action { press }

bright -> off
    action { press }
```

Reusing the terminology introduced in Sec. 3.1, each TML statement consists of a location or edge selector followed by an optional declaration block. Locations have one possible property: `invar` (invariant). Edges have three: `guard`, `action`, and `reset`. Their correspondence to definitions in the TSA model should be obvious.

An edge or location statement's declaration block may have at most one of each property. An automaton must have exactly one initial location, marked using the `init` keyword.

In a program consisting of statements $s_1, s_2, \ldots, s_n$, multiple statements may reference the same location or edge. Each edge statement creates a new edge, while repeated location statements modify an existing location. For each such

pair of location statements s_x, s_y where $x < y$, declarations in s_y have precedence over properties declared in s_x.

We can note that the location statements `low` and `bright` in Lst. 4.1 do not carry any additional information and may be removed. The existence of these locations will be inferred from their appearance in edge statements.

TML is not whitespace-sensitive; indentation is used for readability.

4.2.1 Shallow Features

The following shallow features are fully implemented in the TML compiler, evaluating to standard TSA.

Default Statements

Default properties for locations and edges are implemented following their design in Dot (Sec. 3.1.1), with a location or edge default applying to all statements following its definition.

Default properties may be overridden or cleared locally on a single location or edge statement. A subsequent default statement may also modify or void an existing default property. TML does not have any feature corresponding to Dot's subgraphs, so this voiding mechanism is used for applying defaults to a delimited code block.

Lst. 4.2 below illustrates the details of default statements. On line 2 a default invariant is defined applying to all subsequent location statements. On line 4 the default is overridden with a locally defined invariant on location c. The default is cleared using `!`, locally for one location e on line 6, and for all subsequent locations (i.e., f) on line 7. The resulting model's location invariants are shown in Tbl. 4.1.

Listing 4.2: Example TML program illustrating the use of default properties.

```
1 init a
2 default location invar { x < 1 }
3     b
4     c invar { x < 2 }
5     d
6     e invar!
7 default location invar!
8 f
```

Table 4.1: Result of program in Lst. 4.2 .

Location	Invariant
a	none
b	$x < 1$
c	$x < 2$
d	$x < 1$
e	none
f	none

Selector Shorthands

Mirroring edge selector shorthands in Dot (Sec. 3.1.2), TML lets the user declare several edges at once using *edge chains*:

```
[a, b] -> c -> [d, e]
```

Each link in the chain is either a location or a *location group*, consisting simply of a bracketed list of location identifiers. Like in Dot, properties defined on this selector applies to each generated edge.

Location groups can also be used in location selectors, as a light-weight alternative to defaults:

```
[a, b]
    guard { x < 5 }
```

In Dot the equivalent would be achieved using a location default contained in a subgraph:

```
subgraph {
    node [guard = "x < 5"];
    a; b;
}
```

4.2.2 Deep Features

The following features are not evaluated by the TML compiler, but instead carried over in their original form to the intermediate JSON representation. Thus their exact semantics are decided by the back-end system. These features are considered optional extensions to the TML language, usable in separate language variants (see further Sec. 4.4.2).

Synchronization Actions

The syntax of synchronization actions in TML coincide with Uppaal's input/output actions; i.e., for a channel c, $c!$ is an output action and $c?$ is an input action. Broadcast and urgent channels are not supported.

A network of n automata is modeled using n individual TML files, each describing one automaton. These are compiled separately to n JSON IR files.

Controller Synthesis

To support controller synthesis applications, a TML extension is defined which allows the distinction between controllable and uncontrollable edges. In the view of a system as divided into plant, environment, and controller, this determines which plant transitions are induced by the environment vs. the controller.

The syntax `a -> b` is used for controllable edges, and `a >> b` for uncontrollable edges. This mirrors the implementation in Uppaal Tiga, though Tiga uses the syntax `a -u-> b` for uncontrollable edges. Both types of edges can be used interchangeably within an edge chain.

4.3 Syntax

Due to design challenge C1, the TML compiler is designed to work with a set of syntax variations. Each language variant consists of a common core syntax joined with some set of deep features.

In particular, the compiler supports four language variants:

V1: **TML-TSA**. This is the base language variant, which is fully compatible with TSA. It includes all shallow features of TML.

V2: **TML-SYNC**. This variant is equivalent to TML-TSA, but with the addition of synchronization actions. For compatibility with Uppaal, in this variant actions are used exclusively for communication, i.e., all actions are either input or output actions.

V3: **TML-CTRL**. TML-TSA with the controller synthesis feature described in Sec. 4.2.2.

V4: **TML-CTRL-SYNC**. This is the union of TML-SYNC and TML-CTRL, supporting both synchronization actions and controllable/uncontrollable edges.

The syntax of TML-TSA is listed in an extended Backus-Naur format (BNF) in Lst. 4.3. The starting symbol for the grammar is `Program`. A precise definition of the BNF format used is given in W3C's XML specification [12].

The grammar allows for some semantically incorrect TML programs. We thus add the following validity constraints:

R1 There must be exactly one initial location.

R2 Each statement must have at most one of each type of property.

Listing 4.3: The complete syntax of TML-TSA.

```
Program          ::= statement*

statement        ::= location | edge | default

location         ::= "init"? locationSelector locationProperty*
edge             ::= locationSelector ("->" locationSelector)+
                     edgeProperty*
default          ::= locationDefault | edgeDefault

locationSelector ::= id | "[" id ("," id)* "]"

locationDefault  ::= "default" "location" locationProperty*
edgeDefault      ::= "default" "edge" edgeProperty*

locationProperty ::= invar
edgeProperty     ::= guard | action | reset

invar            ::= "invar" ("!" | "{" invarExpr "}")
guard            ::= "guard" ("!" | "{" guardExpr "}")
action           ::= "action" ("!" | "{" id "}")
reset            ::= "reset" ("!" | "{" clocks "}")

invarExpr        ::= invarConjunct ("&" invarConjunct)*
invarConjunct    ::= id ("<=" | "<") nat

guardExpr        ::= guardConjunct ("&" guardConjunct)*
guardConjunct    ::= (id op nat) | (id "-" id op nat)
op               ::= "<=" | "<" | "==" | ">" | ">="

clocks           ::= id ("," id)*

id               ::= (letter | "_") (letter | "_" | digit)*
letter           ::= [a-zA-Z]
digit            ::= [0-9]
nat              ::= [1-9] digit*
```

Syntax variations in TML-SYNC and TML-CTRL are easily defined by the replacement of rules.

In TML-SYNC, the rule for `action` uses the keyword `sync` and requires `!` or `?` to follow a channel identifier:

```
action ::= "sync" ("!" | "{" id ("!" | "?") "}")
```

TML-CTRL extends the rule for `edge` to allow for uncontrollable edges:

```
edge ::= locationSelector (("->" | ">>") locationSelector)+
         edgeProperty*
```

Finally, TML-CTRL-SYNC is defined by Lst. 4.3 with both of these rule replacements.

As a consequence of the differing feature sets, each language variant also compiles to a distinct IR format. The details are given in Appendix A.

4.4 Implementation

The TML compiler[1] is written entirely in **MCore** [20], a metalanguage designed specifically for use in the Miking project. MCore is a minimal language, intended in the long term to be used primarily as an intermediate language. This multi-level approach is intended to enable the creation of multiple higher-level metalanguages, specializing in the implementation of different types of object languages. The Miking project is currently in pre-release development, so much of the system is liable to change after the writing of this thesis.

As an evaluation of the Miking system, the results from this project are ultimately deemed inconclusive. In order to make definitive judgements about the system in its current form, more time would be needed both for a deeper study of Miking and to establish a framework for comparison with other compiler implementation methods.

4.4.1 Compiler Phases

Compilation of a TML model to JSON IR is implemented in five distinct phases:

P1 Parsing

P2 Semantic analysis

P3 AST transformation

P4 Evaluation

P5 Code generation

TML's parser is constructed using a standard library of parser combinator functions included in the Miking distribution. The parser creates an abstract syntax tree which is structurally very similar to the BNF representation of TML's syntax (Lst. 4.3). Each node below the root of this tree represents a TML statement, i.e., a location, edge, or default statement.

In phase P2, the AST is checked for violations of validity constraints to catch errors in programs which are syntactically correct but semantically ill-formed.

[1] The TML compiler is avaliable online at `https://github.com/ernstwi/tml`. A snapshot of the repository as of 2021/02/26 is embedded in this PDF as an attachment `tml.zip`.

For example, even though any location statement may be preceded by the `init` keyword, by validity constraint R1 a TML program must contain exactly one initial location.

After semantic checks have passed, the AST is transformed to a form which is easier to work with for the following evaluation step. Evaluation is the process of producing an actual timed automata model from the AST representation of a program, through interpretation of location, edge, and default statements.

Finally, this timed automaton model is converted to its intermediate text-based representation using a set of JSON functions from the Miking standard library.

4.4.2 Language Variants

The motivation for using a discrete syntax for each language variant, over simply expanding a single common syntax, is twofold. Firstly, the chosen approach allows for greater freedom in designing the syntax of individual language variants, as a syntax construction in variant A does not need to agree with those in B.

Secondly, using distinct syntaxes lets the compiler catch compatibility errors as early as possible. For instance, in the TML-TSA language variant synchronization actions do not exist, so `sync { a! }` is recognized immediately as a syntax error. If a shared syntax was used, compatibility errors like this would need to be considered explicitly in the semantic analysis phase.

To implement TML language variants, Miking's concept of *language fragments* was used. In this framework, a language fragment is defined as a collection of related data types (generally representing AST nodes) and functions over these. Language fragments are composable, allowing for the definition of a single data type or function to be constructed from multiple merged fragments.

This design is aimed at supporting the construction of extendable compilers, encouraging the implementation of languages in a modular manner as compositions of smaller language fragments.

For the TML compiler, individual language fragments are used for each syntactic element which differs between language variants. For example, two fragments are used to define the different versions of edge actions used in TML-TSA and TML-SYNC. To form complete compilers, these small language fragments are composed with a larger base fragment implementing a general AST used across all TML variants.

4.4.3 Uppaal XTA Adapter

This extension of the TML compiler translates a set of intermediate representation files produced from the TML-SYNC language variant to a model in Uppaal's XTA format. TML-SYNC is used as all its features are compatible with Uppaal.

The adapter program is written separately and in a different programming language to demonstrate the intended use of the IR format, which can be trivially parsed in any programming language with JSON support.

Listing 4.4: Adapter program `ir-to-xta` is used to translate TML-SYNC IR to Uppaal XTA.

```
#!/usr/bin/env node

// This program reads TML-SYNC IR and outputs Uppaal XTA.
// Usage: ./ir-to-xta <IR-1> <IR-2> ... <IR-n>

let fs = require('fs');

let files = process.argv.splice(2);
let irs = files.map(f => JSON.parse(fs.readFileSync(f)));
let channels = new Set();
irs.forEach(ir => ir.channels.forEach(c => channels.add(c)));
files.forEach((f, i) => irs[i].name = f.replace(/\.[^\.]*$/, ''));

print(0, `chan ${[...channels].join(', ')};`);
newline();

irs.forEach(ir => {
  print(0, `process ${ir.name}() {`);

  if (ir.clocks.length > 0) {
    print(4, `clock ${ir.clocks.join(', ')};`);
    newline();
  }
  print(4, 'state');

  ir.locations.forEach(l => {
    write(8, l.id + (l.invariant == null ? '' : ` { ${l.invariant} }`));
    if (last(ir.locations, l))
      print(0, ';')
    else
      print(0, ',')
  });
  newline();

  print(4, `init ${ir.locations.filter(l => l.initial == true)[0].id};`);
  newline();

  if (ir.edges.length > 0) {
    print(4, 'trans');

    ir.edges.forEach(e => {
      print(8, `${e.from} -> ${e.to} {`);
      if (e.guard != null)
        print(12, `guard ${e.guard};`);
      if (e.sync != null)
        print(12, `sync ${e.sync.id}${e.sync.type == 'input' ? '?' :
    '!'};`);
      if (e.reset != null)
        print(12, `assign ${e.reset.map(c => `${c} := 0`).join(', ')};`);
      if (last(ir.edges, e))
        print(8, '};');
      else
```

```
        print(8, '},');
    });
  }
  print(0, '}');
  newline();
});

print(0, `system ${irs.map(ir => ir.name).join(', ')};`);

function print(indent, string) {
  console.log(' '.repeat(indent) + string);
}

function write(indent, string) {
  process.stdout.write(' '.repeat(indent) + string);
}

function last(array, element) {
  if (array.indexOf(element) == array.length - 1)
    return true;
  return false;
}

function newline() {
  process.stdout.write('\n');
}
```

4.5 Summary

To address issues of underspecification, TML is designed with a set of optional extensions building on a shared core syntax. Default statements and selector shorthands are two features which are adopted from Graphviz Dot.

A compiler for TML is implemented, targeting an intermediate JSON representation of timed automata. As a demonstrative use of this IR in a back-end system, a separate program is presented which translates the IR to Uppaal XTA.

Chapter 5

Evaluation

As a basis for evaluation of TML, two example systems are modeled in TML, Kronos, and Uppaal XTA.

The chosen example systems are **Fischer's mutual exclusion protocol** and **CSMA/CD**. Both are protocols from the parallel computing field. Due to their frequent use as benchmarking examples for model checking tools [19], implementations of these protocols are available from authors directly associated with Uppaal and Kronos respectively. The evaluation is based on these existing implementations where possible.

Each protocol is given its own section, wherein the protocol is first explained, after which implementations are presented. Each section is concluded with a brief qualitative assessment of the different implementations' relative merits.

As an aside from the main evaluation, Sec. 5.4 presents an additional XTA model of Fischer's protocol, generated by the TML compiler.

5.1 Method

Prior to performing the evaluation, some conclusions about Kronos and Uppaal XTA were drawn from their examination in chapter 3 which affected the formulation of the core research question.

Kronos' modeling language is very sparse. Some seemingly obvious opportunities for improvement of the user experience are left unimplemented. For example, the number of locations and edges used could easily be inferred from a program instead of requiring this information to be listed in a file header. This leads us to believe that the design of the modeling language itself was likely not a high priority in the Kronos project.

Uppaal on the other hand is highly developed in terms of language features, to a degree which would be difficult to match for any project on the scale of a Master's thesis project.

With this in mind, for an evaluation focused on overall language quality and capabilities, the results seem predetermined.

A choice was thus made to put the question of "Which is best?" aside and focus instead on individual language features. In this more granular view, we consider for both of the studied protocols simply which language features were useful, across all three modeling languages.

The aim of this approach is to reveal firstly whether there is anything new and worthwhile in TML as compared with Uppaal and Kronos, and secondly whether there are specific features from Uppaal or Kronos which might be beneficial as future additions to TML.

5.2 Fischer's Mutual Exclusion Protocol

The purpose of a mutual exclusion protocol is to protect access to some shared resource among n concurrent processes, such that the resource is only accessed by one process at a time. Access to the shared resource is wrapped in a block of code designated as a *critical section*. A mutual exclusion protocol guarantees that only one process at a time is in such a critical section.

This is commonly achieved using some type of locking mechanism. In this terminology, a shared lock is held by at most one process at any point in time. Only the process holding the lock may enter its critical section, and it can not release the lock before the critical section is exited.

Fischer's mutual exclusion protocol [18] uses one shared (atomic) variable id as a lock, shared between n processes $p_1, p_2, \ldots, p_n$.

The protocol is described in pseudocode in Lst. 5.1. When process p_i wants to enter its critical section, it waits until the lock is free ($\mathsf{id} = 0$) and then (after a maximum delay l) requests the lock by setting $\mathsf{id} := i$. p_i then waits a minimum duration k and checks if id is still i. If this is true then p_i enters the critical section, after the conclusion of which it releases the lock by resetting id to 0. If it is false, this means that between line 1 and 4 of process p_i, some other process p_j read $\mathsf{id} = 0$ and requested the lock by setting $\mathsf{id} := j$. In this case p_i will wait until the lock is released, and then try again.

Listing 5.1: Fischer's mutual exclusion protocol for process i.

```
1  wait until id == 0
2      id := i
3      wait k
4      if id == i
5          execute critical section
6          id := 0
7      else
8          go to line 1
```

In the following implementations, we consider Fischer's mutual exclusion protocol for two processes.

5.2.1 Uppaal XTA

The Uppaal XTA implementation presented here is based on a model included as an example with the most recent Uppaal distribution.[25] Automaton P_i^U in Fig. 5.1 describes the behaviour of process p_i.

The location req corresponds to the point between line 1 and 2 of Lst. 5.1, where id has been read but not yet written. The location invariant $x \leq l$ ensures that the automaton stays in this location for at most l time units, which together with the minimum delay k in location wait (enforced by guards on the outgoing edges) is important for the protocol's correctness.

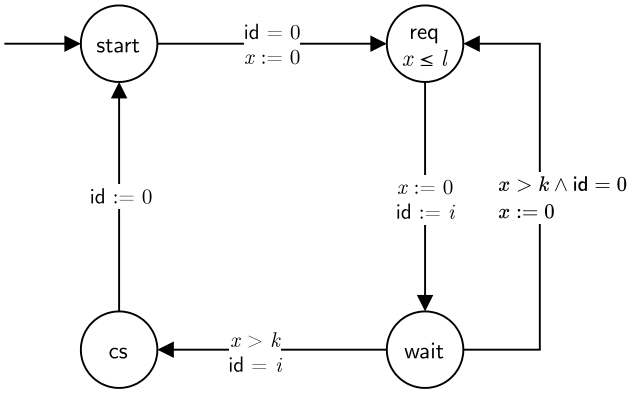


Figure 5.1: Timed automaton P_i^U describing the behaviour of process p_i in Fischer's mutual exclusion protocol.

For the XTA representation of P_i^U, see Lst. 5.2. Note that the template `P` is instantiated on line 37 without a parameter `i`. As described in Sec. 3.2.1, Uppaal will automatically create one process from the template with each possible free parameter value. In this case, since we are using a custom type `id_t` which is an integer ranging over $[1, 2]$, two processes are created.

Listing 5.2: Uppaal XTA implementation of Fischer's mutual exclusion protocol with two processes.

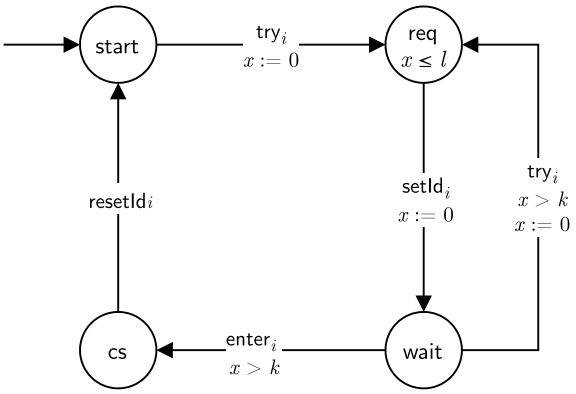
```
1  typedef int[1,2] id_t;
2  int id;
3
4  process P(const id_t i) {
5      clock x;
6      const int k = 10;
7      const int l = 5;
8
9      state
10         wait,
11         req { x<=l },
12         start,
13         cs;
```

```
14      init
15          start;
16
17      trans
18          start -> req {
19              guard id == 0;
20              assign x = 0;
21          },
22          req -> wait {
23              assign x = 0, id = i;
24          },
25          wait -> req {
26              guard x > k && id== 0;
27              assign x = 0;
28          },
29          wait -> cs {
30              guard x>k && id==i;
31          },
32          cs -> start {
33              assign id = 0;
34          };
35  }
36
37  system P;
```

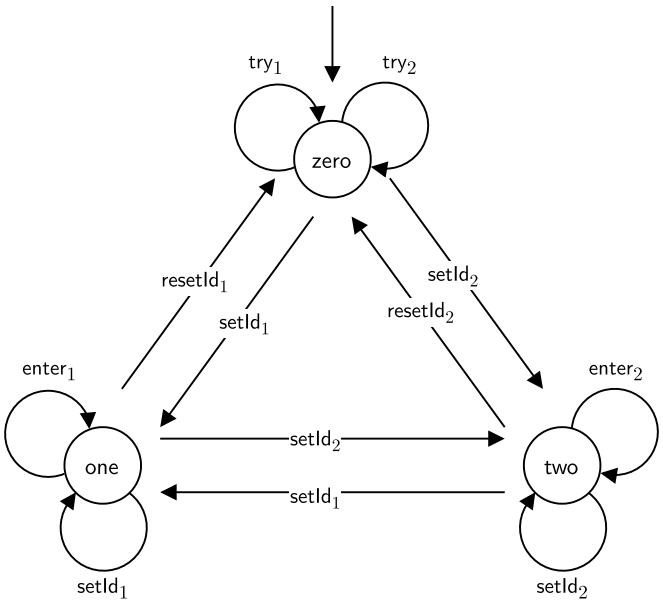
5.2.2 Kronos

Unlike Uppaal, Kronos does not have shared variables, so we must represent id using the only available tool for communication, which is synchronization actions.

To the Uppaal model an automaton V^K is added to simulate the variable id, with each of its locations representing one of id's possible states. This solution is based on an implementation of Fischer's protocol from [11]. It is illustrated in Fig. 5.2 below.



(a) Automaton P_i^K representing process p_i.



(b) Automaton V^K representing the variable id.

Figure 5.2: Kronos model of Fischer's mutual exclusion protocol.

P_1^K and V^K are implemented in Lsts. 5.3, 5.4. Note that we do not have integer constants, so $k = 5$ and $l = 10$ are hard coded. The implementation of P_2^K is omitted – it is identical with P_1^K except for its process identifier.

It should also be noted that technically all automata in a Kronos network share namespaces for locations and clocks, so in reality the location start of P_1^K should be named start$_1$, etc. We choose here to ignore this detail.

Listing 5.3: Kronos implementation of automaton P_1^K from Fig. 5.2a.

```
#locs 4
#trans 5
#clocks x
#sync try_1 setId_1 resetId_1 enter_1

loc: 0
prop: init start
invar: TRUE
trans:
TRUE => try_1; x := 0; goto 1

loc: 1
prop: req
invar: x <= 10
trans:
TRUE => setId_1; x := 0; goto 2

loc: 2
prop: wait
invar: TRUE
trans:
x > 5 => try_1  ; x := 0; goto 1
x > 5 => enter_1;       ; goto 3

loc: 3
prop: cs
invar: TRUE
trans:
TRUE => resetId_1; ; goto 1
```

Listing 5.4: Kronos implementation of automaton V^K from Fig. 5.2b.

```
#locs 3
#trans 12
#clocks
#sync try_1 try_2 setId_1 setId_2 resetId_1 resetId_2 enter_1
    enter_2

loc: 0
prop: init zero
invar: TRUE
trans:
TRUE => setId_1; ; goto 1
TRUE => setId_2; ; goto 2
TRUE => try_1  ; ; goto 0
TRUE => try_2  ; ; goto 0

loc: 1
prop: one
```

```
invar: TRUE
trans:
TRUE => enter_1  ; ; goto 1
TRUE => setId_1  ; ; goto 1
TRUE => setId_2  ; ; goto 2
TRUE => resetId_1; ; goto 0

loc: 2
prop: two
invar: TRUE
trans:
TRUE => enter_2  ; ; goto 2
TRUE => setId_1  ; ; goto 1
TRUE => setId_2  ; ; goto 2
TRUE => resetId_2; ; goto 0
```

5.2.3 TML

With no support for integer variables or constants, the TML implementation of Fischer's protocol (Lsts. 5.5, 5.6) follows Kronos' approach.

Let P_i^T and V^T be automata models equivalent to P_i^K and V^K respectively except that they are adapted to use TML-SYNC's input and output actions. Specifically, all actions on V^T are made into input actions, and all actions on P_i^T are output actions. Reversing this would be equivalent, as all synchronizations in the Kronos model are between V^K and either P_1^K or P_2^K.

Listing 5.5: TML implementation of automaton P_1^T.

```
init start
req invar { x <= 10 }

default edge reset { x }

    start -> req
        sync { try_1! }

    req -> wait
        sync { setId_1! }

    wait -> req
        guard { x > 5 }
        sync { try_1! }

default edge reset!

wait -> cs
    guard { x > 5 }
    sync { enter_1! }
```

```
cs -> start
    sync { resetId_1! }
```

Listing 5.6: TML implementation of automaton V^T.

```
init zero

zero -> zero sync { try_1? }
zero -> zero sync { try_2? }

one -> one sync { enter_1? }
two -> two sync { enter_2? }

one -> zero sync { resetId_1? }
two -> zero sync { resetId_2? }

[zero, two] -> one -> one
    sync { setId_1? }

[zero, one] -> two -> two
    sync { setId_2? }
```

5.2.4 Assessment

Shared integer variables are used to great effect in the XTA implementation of this protocol, replacing the state-keeping automaton necessary in Kronos and TML. The construction used in place of shared integers is quite mechanical and straightforward, but it constitutes an added complexity which must be handled manually by the user. Also, quite crucially, it does not scale well with added processes (larger integer variables).

Uppaal's templates are used to reduce code duplication. In Kronos and TML, additional files not listed but largely identical to the ones given in Lst. 5.3 and Lst. 5.5 respectively are needed to produce a second automaton for process p_2.

It is not clear however whether any of these two features by itself would benefit either the Kronos or TML implementation. In the Uppaal model, templates and shared integer variables are used in combination, with template parameter `i` being used to update `id`.

Furthermore, Uppaal uses integer constants in place of Kronos's and TML's hard coded values for timing variables k and l. Integer constants are useful for readability and maintainability, and their value increases in larger systems.

In the TML implementation of Fischer's protocol, default statements and selector shorthands are used to save some repetition of properties. In Lst. 5.5, an argument could be made that the use of an edge default adds more complexity than what is gained by the absence of repetition, since only applies to three edges. In Lst. 5.6 on the other hand, the benefit of edge selector shorthands seems clear – multiple edges are created in a manner which is concise but still readable.

5.3 CSMA/CD

Carrier sense, multiple access with collision detection is a protocol for coordinating the transmission of messages between multiple senders in a network over a shared bus. We follow the exact formulation from [28], considering a network consisting of two senders s_1, s_2, and modeling only the sending of messages.

When a station wants to use the bus, it first checks whether the bus is already in use, if so delaying a random duration and trying again. When the bus is free, the station starts transmitting its message.

If both senders detects the bus is free within some short time interval, both may start transmitting at once, causing a collision. When this race condition occurs both senders will detect the collision and cancel their transmissions, to retry the transmission procedure after a random duration.

The logic for detecting a collision depends on the worst-case propagation delay σ of the bus network. This is the maximum time it can take for a signal to travel between the outermost nodes of the network, i.e., between s_1 and s_2 in our simple network [22].

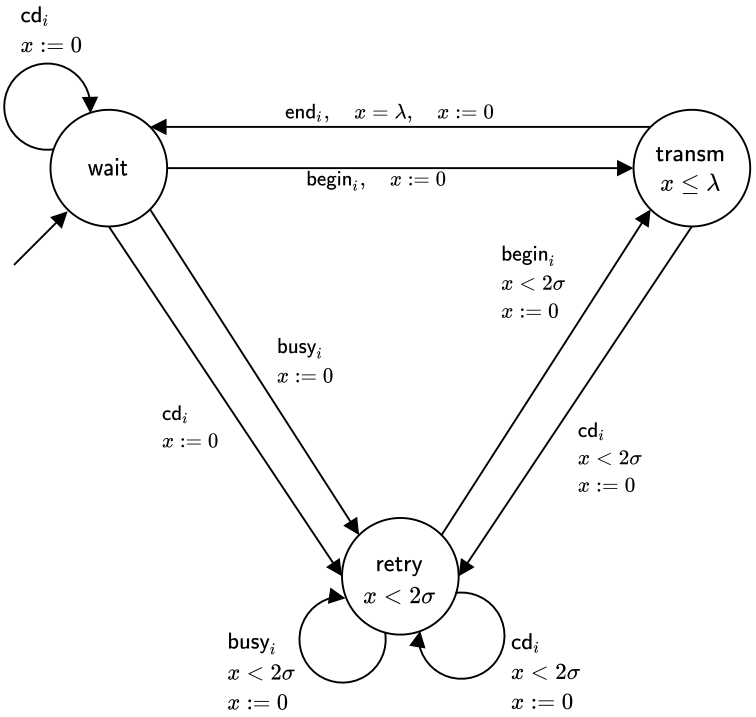
Say that s_1 checks the bus and finds it free, then starts transmitting at time t_0. In the worst-case, the first bit of s_1's message, indicating that the bus is in use, arrives at s_2 at time $t_0 + \sigma$. Assuming that checking the bus's state and starting transmission occurs atomically, s_2 may start transmitting at the latest at $t_0 + \sigma$. There is now a collision, but s_2's signal needs to travel back across the network for s_1 to notice this. s_2's signal will thus arrive back at s_1 at time $t_0 + 2\sigma$.

After s_1 has transmitted for the duration of 2σ without detecting a collision, it is thus guaranteed that no collision will occur and the entire message will be sent.

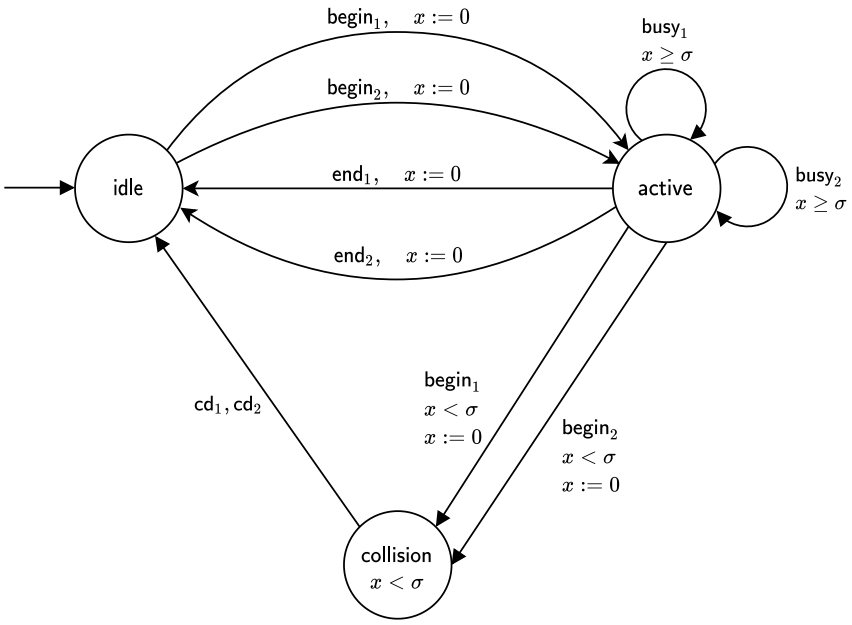
5.3.1 Kronos

Fig. 5.3 illustrates the CSMA/CD protocol in timed automata form. To capture the notion of worst-case propagation delay, we use clock x and constant σ. Λ is the time it takes to send one complete fixed-size message. In the Kronos implementation (Lsts. 5.8, 5.7), $\Lambda = 808$ and $\sigma = 26$ are hard coded.

Both the automata model and its implementation are adapted from [28], with some minor corrections. Specifically, the guard on edge transm $\rightarrow$ retry is changed from $x < \sigma$ to $x < 2\sigma$ to correct what we believe to be an error, supported by [22] and [21]. Furthermore, three discrepancies between automaton figures and corresponding Kronos code have been resolved.



(a) Automaton S_i^K representing sender s_i.



(b) Automaton B^K representing the bus.

Figure 5.3: Kronos model of CSMA/CD.

Listing 5.7: Kronos implementation of automaton S_1^K from Fig. 5.3a.

```
#locs 3
#trans 9
#clocks x
#sync begin_1 end_1 busy_1 cd_1

loc: 0
prop: wait
invar: TRUE
trans:
TRUE => begin_1; x := 0; goto 1
TRUE => busy_1 ; x := 0; goto 2
TRUE => cd_1   ; x := 0; goto 0
TRUE => cd_1   ; x := 0; goto 2

loc: 1
prop: transm
invar: x <= 808
trans:
x = 808 => end_1; x_1 := 0; goto 0
x < 52 => cd_1  ; x_1 := 0; goto 2

loc: 2
prop: retry
invar: x <= 52
trans:
x <= 52 => begin_1; x_1 := 0; goto 1
x <= 52 => busy_1 ; x_1 := 0; goto 2
x <= 52 => cd_1   ; x_1 := 0; goto 2
```

Listing 5.8: Kronos implementation of automaton B^K from Fig. 5.3b.

```
#locs 3
#trans 9
#clocks x
#sync begin_1 begin_2 end_1 end_2 busy_1 busy_2 cd_1 cd_2

loc: 0
prop: idle
invar: TRUE
trans:
TRUE => begin_1; x := 0; goto 1
TRUE => begin_2; x := 0; goto 1

loc: 1
prop: active
invar: TRUE
trans:
x < 26  => begin_1; x := 0; goto 2
x < 26  => begin_2; x := 0; goto 2
```

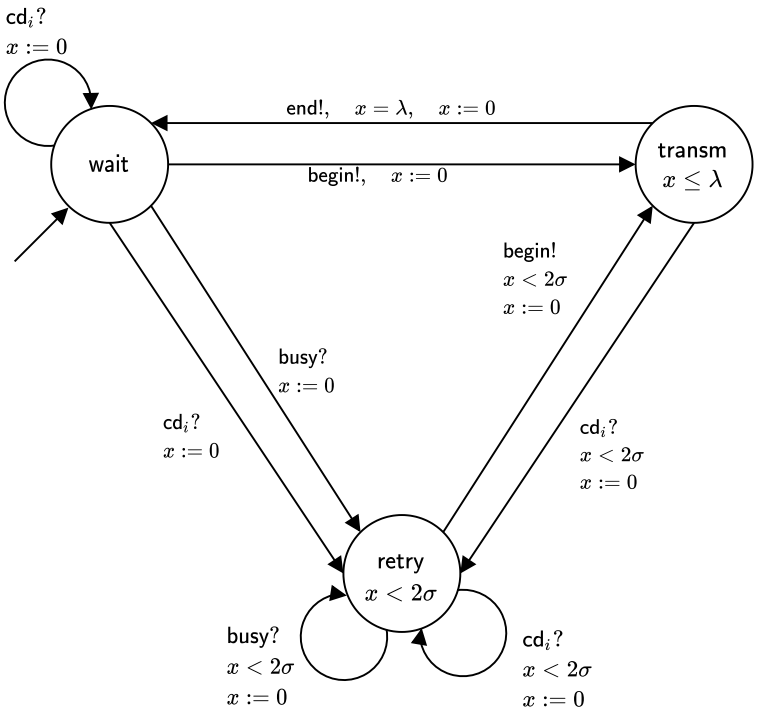
```
x >= 26 => busy_1 ;       ; goto 1
x >= 26 => busy_2 ;       ; goto 1
TRUE    => end_1  ; x := 0; goto 0
TRUE    => end_2  ; x := 0; goto 0

loc: 2
prop: collision
invar: x < 26
trans:
x < 26 => cd_1 cd_2; ; goto 0
```

5.3.2 Uppaal XTA

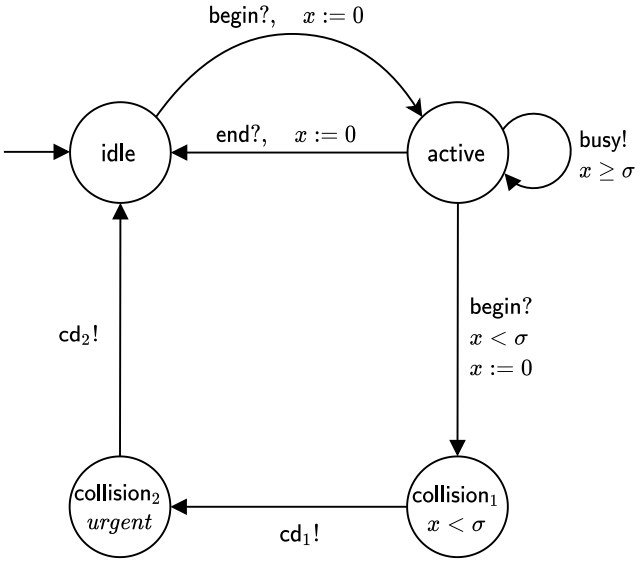
In the Kronos bus automaton B^K (Fig. 5.3b), the edge from collision to idle has two synchronization actions (cd_1, cd_2), making both senders synchronize with the bus at the same instant. In Uppaal, as in TSA, an edge is restricted to one synchronization action.

For the Uppaal-compatible version B^U illustrated in Fig. 5.4b, we solve this by adding an urgent location $\mathsf{collision}_2$. This gives an extra edge $\mathsf{collision}_2 \rightarrow \mathsf{idle}$ which is fired simultaneously with $\mathsf{collision}_2$'s incoming edge (recall that no time may pass in an urgent location). Attaching cd_2 to this extra edge forces the two synchronization actions to be performed in the same time instant.

Uppaal's version of synchronization actions on the other hand allows for some simplification of the bus model as compared with Kronos. In B^K there are four edge pairs which are essentially duplicates of each other. From idle to active, for example, there is one edge synchronizing on begin_1 and another on begin_2, but which are otherwise identical. In B^U, we replace these with a single edge synchronizing on a common input action begin?, and let each sender S_i^U (Fig. 5.4a) use the corresponding output action begin!. The semantics of input and output actions ensures that only one sender synchronizes with the bus each time begin? is performed. The same simplification is used for synchronization actions busy and end.



(a) Automaton S_i^U representing sender s_i in Uppaal.



(b) Automaton B^U representing the bus in Uppaal.

Figure 5.4: Uppaal model of CSMA/CD.

The Uppaal XTA implementation of CSMA/CD (Lst. 5.9) is based on an old implementation made for benchmarking purposes in 2001 [21]. It is one from a set of models with varying numbers of senders, which was generated using an AWK program. As this implementation seems to be based on the earlier Kronos version from [28], the corrections mentioned in Sec. 5.3.1 carry over to the XTA implementation presented here.

For this report the model has also been updated to make use of newer Uppaal features in order to better represent the modeling capabilities of Uppaal today.

Template parameters are used to create multiple senders, removing the need for AWK. To make each sender listen for its specific collision detection action, we define a global array of channels cd and use the process id (template parameter i) to index into this array.

Another update is the introduction of urgent location $\mathsf{collision}_2$. Urgent locations did not exist at the writing of the original implementation, so the same idea was instead implemented using a location invariant $x \leq 0$ on a clock x which was set to 0 on $\mathsf{collision}_2$'s incoming edge. This is an equivalent solution, allowing no time to pass in $\mathsf{collision}_2$.

Listing 5.9: Uppaal XTA implementation of the CSMA/CD model from Fig. 5.4.

```
chan begin, end, busy;
chan cd[2];
const int lambda = 808;
const int sigma = 26;

typedef int[0,1] id_t;
int id;

process Sender(const id_t i) {
    clock x;

    state
        wait,
        transm { x <= lambda },
        retry { x < 2 * sigma };

    init wait;

    trans
        wait -> transm {
            sync begin!;
            assign x := 0;
        },
        wait -> wait {
            sync cd[i]?;
            assign x := 0;
        },
        wait -> retry {
            sync cd[i]?;
```

```
            assign x := 0;
        },
        wait -> retry {
            sync busy?;
            assign x := 0;
        },
        transm -> wait {
            guard x == lambda;
            sync end!;
            assign x := 0;
        },
        transm -> retry {
            guard x < 2 * sigma;
            sync cd[i]?;
            assign x := 0;
        },
        retry -> transm {
            guard x < 2 * sigma;
            sync begin!;
            assign x := 0;
        },
        retry -> retry {
            guard x < 2 * sigma;
            sync cd[i]?;
            assign x := 0;
        },
        retry -> retry {
            guard x < 2 * sigma;
            sync busy?;
            assign x := 0;
        };
}

process Bus() {
    clock x;

    state
        idle,
        active,
        collision_1 { x < sigma },
        collision_2;

    init idle;
    urgent collision_2;

    trans
        idle -> active {
            sync begin?;
            assign x := 0;
        },
```

```
        active -> idle {
            sync end?;
            assign x := 0;
        },
        active -> active {
            guard x >= sigma;
            sync busy!;
        },
        active -> collision_1 {
            guard x < sigma;
            sync begin?;
            assign x := 0;
        },
        collision_1 -> collision_2 {
            sync cd[0]!;
        },
        collision_2 -> idle {
            sync cd[1]!;
        };
}

system Sender, Bus;
```

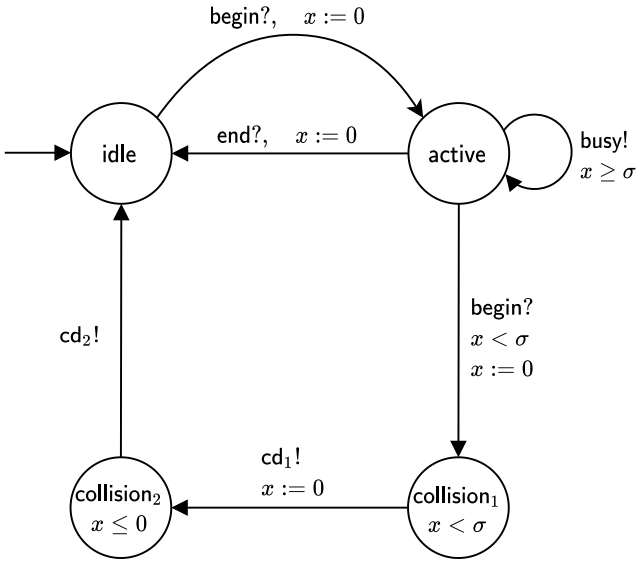


Figure 5.5: Automaton B^T representing the bus in TML.

5.3.3 TML

Like Uppaal, TML is restricted to one synchronization action per edge. To solve the problem of performing $\mathsf{cd}_1!$ and $\mathsf{cd}_2!$ simultaneously, we use the previously described location invariant $x \leq 0$ on $\mathsf{collision}_2$ to simulate an urgent location (see Fig. 5.5).

The sender model S_i^U is used without alterations. As in the Kronos implementation, $\Lambda = 808$ and $\sigma = 26$ are hard coded.

Listing 5.10: TML implementation of automaton S_1^U from Fig. 5.4a.

```
init wait
transm invar { x <= 808 }
retry invar { x <= 52 }

default edge reset { x }

    wait -> wait
        sync { cd_1? }

    wait -> transm
        sync { begin! }

    wait -> retry
        sync { busy? }

    wait -> retry
        sync { cd_1? }

    transm -> wait
        guard { x == 808 }
        sync { end! }

    retry -> transm
        guard { x < 52 }
        sync { begin! }

    transm -> retry -> retry
        guard { x < 52 }
        sync { cd_1? }

    retry -> retry
        guard { x < 52 }
        sync { busy? }
```

Listing 5.11: TML implementation of automaton B^T from Fig. 5.5.

```
init idle
collision_1 invar { x < 26 }
collision_2 invar { x <= 0 }

active -> active
    guard { x >= 26 }
    sync { busy! }

collision_2 -> idle
    sync { cd2! }

default edge reset { x }

    idle -> active
        sync { begin? }

    active -> idle
        sync { end? }

    active -> collision_1
        guard { x < 26 }
        sync { begin? }

    collision_1 -> collision_2
        sync { cd1! }
```

5.3.4 Assessment

For this protocol, the Uppaal and TML implementations have some complexity associated with performing two actions simultaneously. The solution in both cases is to add a location which is restricted such that the system may not rest there for any duration of time. In Uppaal this is slightly simpler to do than in TML thanks to the support for urgent locations. In Kronos this problem does not exist, as multiple actions can be attached to a single edge.

Kronos' synchronization actions, on the other hand, are less suited to CSMA/CD compared to the input/output actions of Uppaal and TML. Where Uppaal and TML can use a single channel (e.g., begin) to communicate between the bus and any number of senders, Kronos requires a unique synchronization action with a corresponding edge on the bus to be added for each sender.

Like in Fischer's protocol, the usefulness of Uppaal's process templates is also evident here. Both Kronos and TML require separate files for senders s_1 and s_2, with everything except the use of cd_1 vs. cd_2 duplicated.

Finally, we observe that TML's default statements are useful in this model, as most edges share the property `reset { x }`.

5.4 Generated XTA

Running the TML compiler with Uppaal XTA adapter on the model of Fischer's protocol from Sec. 5.2.3 produces the XTA model in Lst. 5.12 below. Note that as in the TML source code, the implementation of a second sender is omitted.

Let T be the source TML model of Fischer's protocol (Lsts. 5.5, 5.6) and let X_c be the generated XTA model (Lst. 5.12). Let X_h be the handwritten XTA model (Lst. 5.2).

X_c can be verified as a syntactically valid XTA model by using it as input to Uppaal. By visual inspection X_c is semantically equivalent to T.

Compared to X_h, X_c is clearly more verbose. This is not surprising, and stems from the lack of support in TML for more advanced Uppaal features used in X_h, as explained in Sec. 5.2.3.

Listing 5.12: Uppaal XTA implementation of Fischer's mutual exclusion protocol, compiled from TML Lsts. 5.5, 5.6.

```
chan enter_1, resetId_1, setId_1, try_1, enter_2, resetId_2,
    setId_2, try_2;

process P_T_1() {
    clock x;

    state
        cs,
        req { x<=10 },
        start,
        wait;

    init start;

    trans
        cs -> start {
            sync resetId_1!;
        },
        req -> wait {
            sync setId_1!;
            assign x := 0;
        },
        start -> req {
            sync try_1!;
            assign x := 0;
        },
        wait -> cs {
            guard x>5;
            sync enter_1!;
        },
        wait -> req {
            guard x>5;
```

```
            sync try_1!;
            assign x := 0;
        };
}

process V_T() {
    state
        one,
        two,
        zero;

    init zero;

    trans
        one -> one {
            sync enter_1?;
        },
        one -> one {
            sync setId_1?;
        },
        one -> two {
            sync setId_2?;
        },
        one -> zero {
            sync resetId_1?;
        },
        two -> one {
            sync setId_1?;
        },
        two -> two {
            sync enter_2?;
        },
        two -> two {
            sync setId_2?;
        },
        two -> zero {
            sync resetId_2?;
        },
        zero -> one {
            sync setId_1?;
        },
        zero -> two {
            sync setId_2?;
        },
        zero -> zero {
            sync try_1?;
        },
        zero -> zero {
            sync try_2?;
        };
```

```
}

system P_T_1, V_T;
```

5.5 Summary

In this chapter, we compared implementations of Fischer's protocol and CSMA/CD in Uppaal XTA, Kronos, and TML.

Uppaal's automaton templates were found to be useful in both systems, as both are modeled using a number of nearly identical automata. For Fischer's protocol, Uppaal's shared integer variables were further found to simplify the implementation significantly.

In the CSMA/CD model, Kronos's ability to attach several actions to a single edge was found useful. The Kronos model of synchronization actions was found to be less flexible than that of Uppaal and TML in the modeling of one-to-many communication.

TML's default statements and selector shorthands were used in both models. Both features are used to produce multiple edges or locations with some shared property.

Finally, by presentation of a compiled model of Fischer's protocol, TML is shown to be usable as an input method for Uppaal. Compared to the corresponding handwritten Uppaal model, the compiled model is more complex. This is due to the lack of support in TML for many Uppaal features.

Chapter 6

Discussion

From the evaluation, it is clear that Uppaal XTA supports a great number of useful features not present in either Kronos or TML.

The XTA implementations of Fischer's protocol and CSMA/CD both make use of parameterized templates to create multiple versions of the same process type, e.g., senders in CSMA/CD. Writing multiple such versions manually is time-consuming and makes editing very cumbersome as it results in large amounts of duplicated code.

In addition, shared integer variables seem to be critical for implementing Fischer's protocol in any concise manner.

Shared variables and templates are particularly useful in combination. In Fischer's protocol, for example, we use template parameters to provide each generated automaton process with a process identifier which is used to update a shared variable `id`. In the CSMA/CD implementation, a process identifier is used to index into a shared array of synchronization channels.

The main contributions of TML in terms of language features are default statements and selector shorthands (edge chains and location groups). Both are adapted from Graphviz Dot, with the addition of allowing location groups to be used as selectors in location statements. From the evaluation results, both these features appear useful in a timed automata modeling context.

Edge chains and location groups offer methods for grouping related edges or locations, in a manner which makes the code more concise. Use of such groupings could also improve readability, as they may help convey a better idea of the model's structure.

Like selector shorthands, default statements target groups of edges or locations related via a set of shared properties with the goal of reducing code duplication. From the evaluation, we find that the readability argument for default statements is not as strong as for selector shorthands. This is primarily due to the fact that the distance between a default statement and an element it applies to may be arbitrarily large. This could make it difficult to immediately discern which defaults apply to a given element. The problem can be mitigated somewhat

through the use of indentation, as is done in the evaluation (e.g., Lst. 5.5). Nevertheless, default statements seem at least capable of producing hard-to-read code if not used sensibly, which is a negative.

Finally, the loose requirements that TML places on declarations also affect readability. In particular, locations do not need to be explicitly declared before their use in edge selectors. A location declaration is usually added only in combination with an associated `init` or `invar` keyword, i.e., when the declaration itself adds semantic information. Clocks cannot be explicitly declared at all, as in the TSA model all clocks are initialized to zero at the start of an automaton run.

The general design principle behind these decisions was to make the language as concise as possible, i.e., to avoid forcing the user to write any code which is not strictly needed. Of course, the value of a modeling language does not correlate directly to the number of words required to implement a given model. In this case, one can discuss whether it would have been wiser to follow the convention of requiring variables to be declared before they are referenced, which is used in Kronos and Uppaal as well as in most general purpose programming languages.

Chapter 7

Conclusion and Future Work

In this chapter, we conclude the thesis by relating our main results to the original research questions, and giving some ideas for further development of TML.

7.1 Conclusion

This thesis proposes the language TML as a tool for constructing timed automata models.

From a comparative evaluation of TML against Kronos and Uppaal, Uppaal is considered the most complete due to its well developed feature set. However, TML contributes some new language features which lack direct counterparts in Kronos and Uppaal. Specifically, TML introduces default statements and edge selector shorthands. Both these features are adapted from the general graph modeling language Dot. Through this work, they are shown to be useful in a timed automata modeling context. Consequently, TML is considered to achieve some improvement over the modeling languages of Kronos and Uppaal.

The design of TML is primarily guided by its intended use as an interface for the m2mc system. Though the connection between TML and m2mc is not completely established in this work, it is expected that TML will fit m2mc well. Future development of m2mc may require additions to TML's syntax, though modification of existing TML features should not be necessary.

An additional contribution of this work is a compiler for TML, implemented using the Miking metalanguage system. This was originally intended to answer a secondary research question about the usability of Miking for compiler construction. However, we ultimately consider this work insufficient for providing an accurate evaluation of Miking.

7.2 Future Work

Coupled with the XTA adapter, TML is shown to be usable beyond its intended purpose of integration with m2mc. However, its practical usability for construction of Uppaal models is limited by the large number of Uppaal features not supported in TML. Further development of the TML compiler in this direction could make TML useful as an alternative Uppaal front end.

Aside from the addition of Uppaal features, many of which could be ported more or less directly to TML, we will mention two ideas for improvement of the current compiler implementation.

7.2.1 Disconnected Chains in Edge Selectors

A current restriction on edge selectors is that all locations or location groups are connected in a continuous chain. It makes sense to extend edge selectors so that they can be composed of multiple disconnected chains:

```
[a, b] -> c -> d, e -> f
```

This extension would bring greater parity with location statements, which places no structural requirements on the set of locations used in a selector.

7.2.2 Default Blocks

TML could also benefit from the implementation of some version of Dot's subgraphs (Sec. 3.1.1), used to limit the scope of default statements.

As the similarities with Graphviz Dot increase in number, one might also consider the alternative of using Dot directly as the input language for a timed automata system. The generality of its syntax allows for attachment of a named string or integer value to any location or edge, which could be used to represent timed automaton properties like location invariants and edge guards. This is exemplified in Sec. 3.1, Lsts. 3.1, 3.2.

Bibliography

[1] Alur, R., Courcoubetis, C. and Dill, D. 1990. Model-checking for real-time systems. *Proceedings of the fifth annual IEEE symposium on logic in computer science*. IEEE. 414–425.

[2] Alur, R. and Dill, D.L. 1994. A theory of timed automata. *Theoretical computer science*. Elsevier. 126, 2. 183–235.

[3] Behrmann, G., David, A. and Larsen, K.G. 2004. A tutorial on Uppaal. *Formal methods for the design of real-time systems: Fourth international school on formal methods for the design of computer, communication, and software systems*. Springer. 200–236.

[4] Behrmann, G., David, A. and Larsen, K.G. A tutorial on Uppaal 4.0: *https://www.it.uu.se/research/group/darts/papers/texts/new-tutorial.pdf*. Accessed: 2020-09-21. Revised and extended version of [3].

[5] Bengtsson, J. and Yi, W. 2003. Timed automata: Semantics, algorithms and tools. *Advanced course on petri nets*. Springer. 87–124.

[6] Bouyer, P. et al. 2018. Model checking real-time systems. *Handbook of model checking*. Springer. 1001–1046.

[7] Boyd, S.P. Lecture notes on feedback control systems: *https://stanford.edu/~boyd/ee102/ctrl-static.pdf*. Accessed: 2021-01-05.

[8] Broman, D. 2019. A vision of Miking: Interactive programmatic modeling, sound language composition, and self-learning compilation. *Proceedings of the 12th ACM SIGPLAN international conference on software language engineering*. ACM. 55–60.

[9] Broman, D. High-confidence formal verification of real cyber-physical systems: From models to machine code: *https://people.kth.se/~dbro/projects.html*. Accessed: 2020-09-14.

[10] Cascading style sheets: *https://www.w3.org/Style/CSS*. Accessed: 2021-01-02.

[11] Daws, C. et al. 1995. The tool Kronos. *Lecture notes in computer science: Hybrid systems III, verification and control*. Springer. 208–219.

[12] Extensible markup language (XML) 1.0 (fifth edition): Notation: *https://www.w3.org/TR/REC-xml/#sec-notation*. Accessed: 2021-01-22.

[13] Graphviz: *https://graphviz.org*. Accessed: 2021-01-02.

[14] Graphviz: The Dot language: *https://graphviz.org/doc/info/lang.html*. Accessed: 2020-09-30.

[15] Henzinger, T.A. et al. 1994. *Symbolic model checking for real-time systems*. Cornell University.

[16] Jard, C. and Roux, O.H. 2009. *Communicating embedded systems*. Wiley.

[17] Kronos: *http://www-verimag.imag.fr/DIST-TOOLS/TEMPO/kronos*. Accessed: 2020-10-28.

[18] Lamport, L. 1987. A fast mutual exclusion algorithm. *ACM Transactions on Computer Systems*. ACM. 5, 1. 1–11.

[19] Menghi, C. et al. 2020. Model checking MITL formulae on timed automata: A logic-based approach. *ACM Transactions on Computational Logic*. ACM. 21, 3. 1–44.

[20] Miking: README: *https://github.com/miking-lang/miking/blob/develop/README.md*. Accessed: 2020-12-08.

[21] Moeller, M.O. 2001. Uppaal CSMA/CD model: *https://www.it.uu.se/research/group/darts/uppaal/benchmarks/csma/csma_input_02.ta*. Accessed: 2020-12-11.

[22] Nutt, G. and Bayer, D. 1982. Performance of CSMA/CD networks under combined voice and data loads. *IEEE Transactions on Communications*. IEEE. 30, 1. 6–11.

[23] Pombrio, J. and Krishnamurthi, S. 2014. Resugaring: Lifting evaluation sequences through syntactic sugar. *Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation*. ACM. 361–371.

[24] Uppaal: *http://www.uppaal.org*. Accessed: 2020-09-30.

[25] Uppaal 4.1.24 for 64-bit macOS: *http://uppaal.org/download/*. Accessed: 2020-12-02.

[26] Uppaal Tiga: *https://people.cs.aau.dk/~adavid/tiga/*. Accessed: 2020-10-27.

[27] Waez, M.T.B., Dingel, J. and Rudie, K. 2013. A survey of timed automata for the development of real-time systems. *Computer Science Review*. Elsevier. 9. 1–26.

[28] Yovine, S. 1997. Kronos: A verification tool for real-time systems. *International Journal on Software Tools for Technology Transfer*. Springer. 1, 1-2. 123–133.

Appendix A

Intermediate Representation

This appendix details the intermediate JSON representation of each TML variant. The IR for TML-TSA is presented first, after which follows modifications used for the other language variants. For details on the JSON format, see `https://www.json.org`.

TML-TSA

Table A.1: Root object

Property	Type	Description
`actions`	Array of strings	All actions used in the program.
`clocks`	Array of strings	All clocks used in the program.
`locations`	Array of *Location* objects	Automaton locations.
`edges`	Array of *Edge* objects	Automaton edges.

Table A.2: Location object

Property	Type	Description
`id`	String	Location identifier.
`initial`	Boolean	True for the initial location, false for all others.
`invariant`	Null or string	Location invariant (optional).

Table A.3: Edge object

Property	Type	Description
`from`	String	Identifier of source location.
`to`	String	Identifier of target location.
`guard`	Null or string	Guard expression (optional).
`reset`	Null or array of strings	Set of clocks to reset (optional).
`action`	Null or string	Edge action identifier (optional).

TML-SYNC

In this variant, the property `action` on *Edge* objects is replaced with an optional property `sync` of type *Action* object described below. On the root object, the property `actions` is renamed `channels`.

Table A.4: Action object

Property	Type	Description
`id`	String	Action identifier.
`type`	String	`input` or `output`.

TML-CTRL

This variant adds a Boolean property `controllable` to the Edge object. Otherwise the format is identical to that of TML-TSA.

TML-CTRL-SYNC

This variant uses TML-TSA's IR format with the modifications from both TML-SYNC and TML-CTRL.

TRITA-EECS-EX-2021:58